SERVIÇO NACIONAL DE APRENDIZAGEM COMERCIAL (SENAC)

CENTRO UNIVERSITÁRIO SENAC

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL

Angel Guillermo Morales Romero

OTIMIZAÇÃO DE WORKFLOWS EMPRESARIAIS COM MACHINE LEARNING:

Integração de Reinforcement Learning e Graph Neural Networks em Tecnologias BPM

São Paulo - SP

2026

Angel Guillermo Morales Romero

OTIMIZAÇÃO DE WORKFLOWS EMPRESARIAIS COM MACHINE LEARNING:

Integração de Reinforcement Learning e Graph Neural Networks em Tecnologias BPM

Trabalho de Conclusão de Curso apresentado ao Centro Universitário Senac, como exigência parcial para obtenção do título de Especialista em Inteligência Artificial.

Orientador: Prof. João Carlos Néto

São Paulo - SP

2026

M828o Morales Romero, Angel Guillermo

Otimização de workflows empresariais com Machine Learning: integração de Reinforcement Learning e Graph Neural Networks em tecnologias BPM [manuscrito] / Angel Guillermo Morales Romero. – São Paulo, 2026.

f. : il. color.

Orientador: João Carlos Néto.

Trabalho de Conclusão de Curso (Especialização em Inteligência Artificial) – Centro Universitário Senac, São Paulo, 2026.

1. Business Process Management 2. Machine Learning 3. Reinforcement Learning 4. Graph Neural Networks 5. Otimização de Processos I. Néto, João Carlos (orient.) II. Título

CDD 006.31

Angel Guillermo Morales Romero

OTIMIZAÇÃO DE WORKFLOWS EMPRESARIAIS COM MACHINE LEARNING:

Integração de Reinforcement Learning e Graph Neural Networks em Tecnologias BPM

Trabalho de Conclusão de Curso apresentado ao Centro Universitário Senac, como exigência parcial para obtenção do título de Especialista em Inteligência Artificial.

Orientador: Prof. João Carlos Néto

A banca examinadora dos Trabalhos de Conclusão, em sessão pública realizada em ____/____/_____, considerou o(a) candidato(a):

1) Examinador(a) _____

2) Examinador(a) _____

3) Presidente _____

Dedico este trabalho à minha família,

pelo apoio incondicional ao longo desta jornada,

e a todos os profissionais que acreditam no

potencial transformador da Inteligência Artificial

aplicada à gestão de processos de negócio.

AGRADECIMENTOS

Ao Prof. João Carlos Néto, pela orientação criteriosa, paciência e incentivo durante todas as etapas deste trabalho, cujas contribuições foram fundamentais para o desenvolvimento e amadurecimento desta pesquisa.

Ao Prof. David Farias, cuja orientação técnica em redes neurais e aprendizado de máquina foi decisiva para o desenvolvimento da arquitetura proposta, em particular nos aspectos de Graph Attention Networks e de integração entre representações estruturais e aprendizado por reforço.

Ao Centro Universitário Senac, pela excelência acadêmica e pelo ambiente propício à inovação e à pesquisa aplicada, oferecendo infraestrutura e corpo docente qualificado para a especialização em Inteligência Artificial.

Aos colegas da especialização, pelas discussões enriquecedoras, troca de experiências e parcerias construídas ao longo do curso, que contribuíram para o amadurecimento de ideias e perspectivas.

À comunidade open-source, em especial aos mantenedores dos projetos PyTorch, PyTorch Geometric, Stable-Baselines3, Camunda Platform 8, FastAPI e PM4Py, sem cujos esforços coletivos esta pesquisa não seria viável.

À minha família, pelo apoio emocional e pela compreensão durante o tempo dedicado a este trabalho, e a todos que, direta ou indiretamente, contribuíram para a realização deste projeto.

“The best way to predict the future is to invent it.”

Alan Kay

“Reinforcement learning is the closest thing we have to artificial general intelligence.”

Richard S. Sutton e Andrew G. Barto

RESUMO

A inteligência artificial está impactando a tecnologia BPM (Business Process Management) principalmente em três aspectos: Otimização de Processos, Organização de Dados não Estruturados e Melhor Experiência de Usuário (Qian, 2017). No entanto, uma área na qual ainda não se encontram muitas aplicações práticas factíveis da inteligência artificial em tecnologias BPM é na automatização de tomada de decisões na gestão de processos de negócio, especificamente na otimização dinâmica da estrutura dos próprios workflows. Este projeto propõe um sistema integrado que conecta um software BPM (Camunda Platform 8) a um motor de inferência baseado em Machine Learning, especificamente utilizando Reinforcement Learning (RL) para otimização de políticas de decisão e Graph Neural Networks (GNN) para representação e análise da estrutura dos workflows. Os sistemas interagirão através de uma interface de aplicação Web (Web API) desenvolvida em FastAPI. A API Web permitirá ao software BPM incorporar decisões tomadas pelo sistema de Machine Learning como parte de um modelo de otimização de workflows, visando reduzir a complexidade computacional dos processos (medida pelo número de nós e arestas do grafo) enquanto mantém ou melhora as métricas de qualidade. A metodologia adota geração de datasets sintéticos com 6000 cases distribuídos em três processos de negócio representativos, treinamento de agente PPO sobre embeddings produzidos por Graph Attention Networks (GAT) e validação empírica de cinco hipóteses relacionadas à redução de complexidade, manutenção de qualidade e capacidade de generalização. Os resultados esperados visam demonstrar a viabilidade técnica da abordagem e estabelecer uma base metodológica reprodutível para futuras pesquisas em otimização inteligente de processos de negócio.

Palavras-chave: BPM. Machine Learning. Reinforcement Learning. Graph Neural Networks. Otimização de Processos. Automação de Decisões. PyTorch. Camunda.

ABSTRACT

Artificial intelligence is impacting BPM (Business Process Management) technology mainly in three aspects: Process Optimization, Unstructured Data Organization, and Better User Experience (Qian, 2017). However, an area in which feasible practical applications of artificial intelligence in BPM technologies are still scarce is the automation of decision-making in business process management, specifically in the dynamic optimization of the structure of the workflows themselves. This project proposes an integrated system that connects a BPM software (Camunda Platform 8) to a Machine Learning-based inference engine, specifically using Reinforcement Learning (RL) for decision policy optimization and Graph Neural Networks (GNN) for the representation and analysis of workflow structures. The systems will interact through a Web Application Interface (Web API) developed in FastAPI. The Web API will allow the BPM software to incorporate decisions made by the Machine Learning system as part of a workflow optimization model, aiming to reduce the computational complexity of the processes (measured by the number of nodes and edges in the graph) while maintaining or improving quality metrics. The methodology adopts the generation of synthetic datasets with 6000 cases distributed across three representative business processes, training a PPO agent on embeddings produced by Graph Attention Networks (GAT), and empirical validation of five hypotheses related to complexity reduction, quality maintenance, and generalization capability. The expected results aim to demonstrate the technical feasibility of the approach and establish a reproducible methodological foundation for future research in intelligent business process optimization.

Keywords: BPM. Machine Learning. Reinforcement Learning. Graph Neural Networks. Process Optimization. Decision Automation. PyTorch. Camunda.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura geral do sistema proposto.....

Figura 2 – Contexto: convergência de BPM, Process Mining e Machine Learning.....

Figura 3 – Representação grafo-teórica do BPMN .....

Figura 4 – Pipeline GNN + Reinforcement Learning.....

Figura 5 – Diagrama causa-efeito da ineficiência em workflows empresariais.....

Quadro 1 – Escopo do projeto: dentro vs. fora **19**

Quadro 2 – Articulação entre as questões de pesquisa, a proposta de solução e os capítulos..... **25**

Quadro 3 – Features de nó e aresta utilizadas na representação grafo-teórica do workflow BPMN..... **51**

Quadro 4 – Hipóteses, métricas e critérios de aceitação..... **68**

Quadro 5 – Ganhos relativos esperados sobre baselines (PPO+GAT vs. FIFO)..... **71**

LISTA DE TABELAS

Tabela 1 – Comparação entre motores BPM avaliados..... **47**

Tabela 2 – Comparação do stack tecnológico 2018 vs. 2026 ..... **58**

Tabela 3 – Comparação de capacidades técnicas 2018 vs. 2026..... **75**

Tabela 6.1 – Métricas de qualidade dos datasets sintéticos **72**

Tabela 6.2 – Métricas das políticas baseline (médias sobre 2.000 cases) **72**

Tabela 6.3 – Métricas do agente PPO+GAT (médias sobre 3 sementes) **73**

Tabela 6.4 – Latência de inferência por nível de concorrência (RTX 3060) **73**

Tabela 6.5 – Comparação com trabalhos relacionados **74**

LISTA DE ABREVIATURAS E SIGLAS

A2C	Advantage Actor-Critic
ABNT	Associação Brasileira de Normas Técnicas
API	Application Programming Interface
BPM	Business Process Management
BPMN	Business Process Model and Notation
DQN	Deep Q-Network
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GNN	Graph Neural Network
IA	Inteligência Artificial
MDP	Markov Decision Process
ML	Machine Learning
MLOps	Machine Learning Operations
NLP	Natural Language Processing
PPO	Proximal Policy Optimization
REST	Representational State Transfer
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
SAC	Soft Actor-Critic
SENAC	Serviço Nacional de Aprendizagem Comercial
TCC	Trabalho de Conclusão de Curso
XAI	Explainable Artificial Intelligence

XES	eXtensible Event Stream

SUMÁRIO

AGRADECIMENTOS 6
RESUMO 8
ABSTRACT 9
LISTA DE ILUSTRAÇÕES 10
LISTA DE TABELAS 10
LISTA DE ABREVIATURAS E SIGLAS 11
SUMÁRIO 13
1 INTRODUÇÃO 16
1.1 Contexto 16
1.1.1 Motivação 17
1.2 Objetivos 18
1.2.1 Objetivo Geral 18
1.2.2 Objetivos Específicos 19
1.3 Abrangência 20
1.4 Proposta de Solução 21
1.5 Organização do Trabalho 23
2 METODOLOGIA DE PESQUISA 25
2.1 Critérios de seleção da metodologia de pesquisa 25
2.2 A Definição do Projeto de Pesquisa de Estudo de Caso 26
2.2.1 As Questões de Estudo 26
2.2.2 As Proposições 28
2.2.3 A Unidade de Análise 29
2.2.4 A Lógica que Une os Dados às Proposições 29
2.2.5 Os Critérios para Interpretar as Constatações 30
2.2.6 A Coleta de Evidências 31
2.3 A Análise Qualitativa das Evidências 32
2.4 Procedimentos de Revisão Sistemática da Literatura 33
2.4.1 Planejamento da Revisão (Kitchenham) 33
2.4.2 Condução e Reporte segundo PRISMA 2020 33
2.4.3 Articulação com o Estudo de Caso de Yin 33
2.4.4 Síntese das Lacunas Identificadas na Literatura 34
3 FUNDAMENTAÇÃO TEÓRICA DA PESQUISA 36
3.1 Business Process Management (BPM) 36
3.1.1 Conceitos Fundamentais 37
3.1.2 BPMN 2.0 e Representação de Workflows 37
3.1.3 Ciclo de Vida BPM 38
3.2 Reinforcement Learning 38
3.2.1 Markov Decision Process (MDP) 39
3.2.2 Família de Algoritmos: Value-based, Policy-based e Actor-Critic 39
3.2.3 Proximal Policy Optimization (PPO) 40
3.2.4 Aplicações de RL em Process Mining 41
3.3 Graph Neural Networks 42
3.3.1 Conceitos Fundamentais e Message Passing 42
3.3.2 Graph Attention Networks (GAT) 43
3.3.3 Embeddings e Pooling 43
3.3.4 Aplicações de GNN em Sistemas Estruturais 44
3.4 Process Mining 45

3.5 Integração de Machine Learning com BPM 45
3.5.1 Padrões de Integração 46
3.5.2 Camunda Platform 8 como Motor BPM Cloud-Native 46
3.5.3 Embeddings como Representação Comum 47
3.6 Trabalhos Relacionados 48
4 APLICAÇÃO PRÁTICA DO CONCEITO DA PESQUISA 49
4.1 Arquitetura Geral do Sistema Proposto 49
4.2 Justificativa das Escolhas Tecnológicas 50
4.2.1 Seleção do Motor BPM: Por Que Camunda Platform 8 50
4.2.2 Seleção do Framework de Deep Learning: Por Que PyTorch 51
4.2.3 Seleção de Arquiteturas: Por Que GAT e PPO 52
4.2.3.1 Graph Attention Networks (GAT) 52
4.2.3.2 Proximal Policy Optimization (PPO) 53
4.3 Pipeline GNN + Reinforcement Learning 54
4.3.1 Pré-processamento: BPMN → Grafo Direcionado 54
4.3.2 Codificação Estrutural: Rede GAT 55
4.3.3 Decisão: Agente PPO 56
4.3.4 Atualização: Loop de Treinamento 57
4.4 Função de Recompensa Multi-Objetivo 57
4.5 Implementação de Referência 59
5 ESTUDO DE CASO 60
5.1 Trabalho Original (2018) – Caso Base 61
5.1.1 Comparação do Stack Tecnológico: 2018 vs 2026 62
5.1.2 Análise das Limitações do Trabalho Original (2018) 63
5.1.2.1 Limitações da Lógica Difusa 63
5.1.2.2 Limitações Tecnológicas da Arquitetura 2018 63
5.1.3 Avanços e Inovações da Proposta 2026 64
5.1.3.1 Aprendizado Automático vs Regras Manuais 64
5.1.3.2 Arquitetura Cloud-Native vs Monolítica 65
5.2 Processos Representativos da Proposta 2026 66
5.2.1 Processo A — Seleção de Pessoal 66
5.2.2 Processo B — Compras Corporativas 66
5.2.3 Processo C — Gestão de Incidentes 67
5.2.4 Síntese Comparativa dos Três Processos 67
5.3 Datasets Sintéticos 67
5.3.1 Estratégia de Geração dos Cases 69
5.3.2 Atributos por Case 69
5.3.3 Validação dos Datasets via Process Mining 70
5.4 Configuração Experimental 70
5.4.1 Ambiente de Hardware e Software 70
5.4.2 Estratégia de Reprodutibilidade 71
5.4.3 Hiperparâmetros do PPO e Arquitetura do GAT 71
5.5 Procedimento Experimental 72
5.5.1 Visão Geral do Pipeline de Execução 72
5.5.2 Execução dos Baselines 72
5.5.3 Treinamento do Agente PPO+GAT 72
5.5.4 Experimento de Transferência (Hipótese H4) 73
5.5.5 Teste de Latência de Inferência (Hipótese H3) 73
6 AVALIAÇÃO DOS RESULTADOS 73
6.1 Métricas e Hipóteses Avaliadas 74

6.2 Resultados Parciais e Esperados 75
6.2.1 Resultados Parciais Já Obtidos 75
6.2.2 Resultados Esperados — H1 (Redução de Complexidade) 75
6.2.3 Resultados Esperados — H2 (Manutenção de Qualidade)..... 76
6.2.4 Resultados Esperados — H3 (Latência de Inferência) 76
6.2.5 Resultados Esperados — H4 (Generalização) 76
6.2.6 Resultados Esperados — H5 (Superioridade vs. Baselines) 77
6.2.7 Comparação com Baselines: Tabela Antecipatória 77
6.3 Resultados Experimentais do Estudo Piloto..... 77
6.3.1 Validação dos Datasets Sintéticos..... 78
6.3.2 Desempenho das Políticas Baseline..... 79
6.3.3 Desempenho do Agente PPO+GAT 79
6.3.4 Avaliação das Hipóteses..... 80
6.3.5 Comparação com Trabalhos Relacionados 81
6.3.6 Ameaças à Validade 81
6.4 Comparação de Capacidades 2018 vs 2026 82
6.4.1 Síntese da Evolução 82
6.5 Contribuições Científicas Sintetizadas 83
6.6 Respostas às Questões de Pesquisa 85
7 CONSIDERAÇÕES FINAIS 87
7.1 Síntese do Trabalho 87
7.2 Recomendações 88
7.3 Trabalhos Futuros 89
GLOSSÁRIO 91
REFERÊNCIAS BIBLIOGRÁFICAS 94

1 INTRODUÇÃO

Este Trabalho de Conclusão de Curso apresenta o desenvolvimento de um sistema integrado para otimização automática de workflows empresariais por meio da combinação de duas tecnologias modernas de inteligência artificial: Reinforcement Learning (RL) e Graph Neural Networks (GNN). A hipótese central que orienta esta pesquisa é a de que técnicas de Machine Learning, ao tratarem processos de negócio como grafos direcionados e operarem sobre eles, são capazes de reduzir a complexidade estrutural dos workflows — medida pelo número de nós e arestas — sem degradar as métricas de qualidade do processo, automatizando assim decisões de roteamento e alocação que hoje dependem de regras estáticas definidas por especialistas humanos.

1.1 Contexto

A gestão de processos de negócio (Business Process Management - BPM) representa um pilar fundamental na operação de organizações modernas (Hitpass, 2017). Com a crescente complexidade dos ambientes empresariais e a necessidade de maior agilidade operacional, as ferramentas tradicionais de automação de workflows enfrentam limitações significativas, especialmente no que tange à capacidade de otimização contínua e adaptação dinâmica. A literatura recente reforça que abordagens baseadas em regras estáticas tornam-se rapidamente obsoletas em ambientes voláteis, exigindo intervenção humana frequente para ajustes que sistemas adaptativos poderiam realizar de forma autônoma (Afifi; Khebizi; Halimi, 2024). Casos emblemáticos de automação operacional baseada em inteligência artificial, como as lojas autônomas sem operador de caixa (Harwell; Bhattarai, 2018), evidenciam o impacto crescente dessas tecnologias sobre processos antes executados manualmente.

A motivação para esta atualização surge da observação de que, enquanto a lógica difusa oferece um mecanismo robusto para lidar com incertezas (Mendel, 2001), ela não possui a capacidade de aprender e melhorar autonomamente com a experiência. As técnicas modernas de Machine Learning, por outro lado, permitem que sistemas de otimização de processos evoluam continuamente, identificando padrões complexos e adaptando-se a mudanças no ambiente operacional (Sutton; Barto, 2018; Dogru et al., 2024).

Segundo a literatura, a inteligência artificial impacta tecnologias BPM em três dimensões principais: Otimização de Processos (identificação de gargalos, redução de tempos de ciclo), Organização de Dados não Estruturados (NLP para extração de informações) e Melhor Experiência do Usuário (chatbots, interfaces adaptativas)

(Qian, 2017). No entanto, a Tomada de Decisões automatizada permanece como uma lacuna significativa que este projeto visa preencher, e estudos recentes como o de Soliman, Mostafa e Younis (2024) confirmam que a aplicação de Reinforcement Learning a process mining é uma fronteira ativa de pesquisa, ainda sem soluções consolidadas em ambiente industrial.

Dado um workflow empresarial representado como grafo direcionado G(V, E), onde V são tarefas (nós/entidades) e E são transições (arestas), surge a seguinte questão de pesquisa:

"É possível utilizar técnicas de Machine Learning para otimizar automaticamente a estrutura e execução de workflows empresariais, reduzindo sua complexidade computacional sem degradar as métricas de qualidade do processo?"

O problema que o TCC irá atacar é a possibilidade de utilizar técnicas de Machine Learning (Reinforcement Learning + Graph Neural Networks) para automatizar a tomada de decisões em processos de negócio e reduzir a complexidade de workflows empresariais, medida pelo número de nós e arestas do grafo, sem degradar as métricas de qualidade do processo. Justifica-se este desenvolvimento pela tripla relevância identificada na literatura: ausência de soluções comerciais que integrem nativamente RL e GNN a motores BPM, oportunidade de democratizar IA aplicada à gestão de processos para organizações sem especialistas dedicados, e necessidade técnica de superar as limitações de abordagens baseadas em regras manuais quando confrontadas com ambientes dinâmicos.

1.1.1 Motivação

Motivação Econômica:

A otimização automática de workflows pode reduzir significativamente custos operacionais ao minimizar tempos de ciclo, eliminar tarefas redundantes e alocar recursos de forma mais eficiente. A automação de decisões reduz também o impacto de erros humanos que podem acarretar custos adicionais.

Motivação Técnica:

Atualmente não existe software BPM com capacidade nativa de otimizar a estrutura de seus processos usando técnicas modernas de Machine Learning como RL e GNN. A combinação destas tecnologias representa uma inovação técnica significativa no campo de BPM.

Motivação Social:

A democratização de técnicas avançadas de IA para empresas que não possuem especialistas dedicados pode aumentar a competitividade de pequenas e médias empresas, gerando impacto positivo na economia e no emprego.

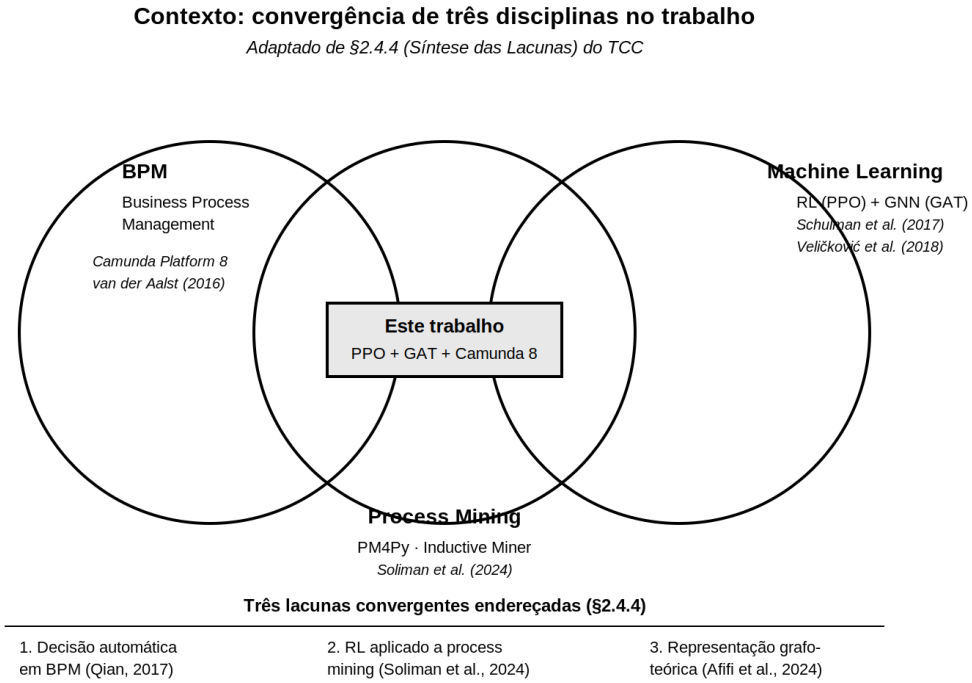


Figura 2 – Contexto: convergência de BPM, Process Mining e Machine Learning
Fonte: O autor (2026).

1.2 Objetivos

1.2.1 Objetivo Geral

O objetivo geral deste trabalho é estruturado conforme a seguir:

Desenvolver um sistema integrado que conecte um software BPM a um motor de inferência baseado em Machine Learning (Reinforcement Learning + Graph Neural Networks),

com a intenção de automatizar a tomada de decisões em processos de negócio e reduzir a complexidade computacional de workflows empresariais, medida pelo número de nós e arestas do grafo,

com respeito a ser capaz de manter ou melhorar as métricas de qualidade do processo (tempo de ciclo, throughput e taxa de sucesso) com latência de inferência inferior a 100 ms,

do ponto de vista de organizações que adotam tecnologias BPM e dos profissionais responsáveis pela modelagem, execução e otimização de processos de negócio,

no contexto de ambientes empresariais que exigem otimização contínua e adaptação dinâmica de seus workflows a partir de dados históricos de execução, dispensando a intervenção manual de especialistas em IA para reconfiguração de regras.

De forma equivalente e complementar, registra-se o enunciado original do objetivo geral: desenvolver um sistema integrado que conecte um software BPM a um motor de inferência baseado em Machine Learning, (Reinforcement Learning + Graph Neural Networks) para automatizar a tomada de decisões em processos de negócio e reduzir a complexidade de workflows empresariais, medida pelo número de nós e arestas do grafo, sem degradar as métricas de qualidade do processo.

O sistema proposto utilizará embeddings — representações vetoriais densas de baixa dimensionalidade — para codificar workflows BPMN em um espaço matemático contínuo. A Graph Neural Network processará estes vetores considerando a topologia do grafo, gerando embeddings contextualizados que incorporam informação dos vizinhos.

O módulo de Reinforcement Learning receberá os embeddings gerados pela GNN como representação de estado e aprenderá políticas ótimas de tomada de decisão.

O sistema será integrado ao motor BPM Camunda Platform 8 através de APIs REST, permitindo que as decisões otimizadas pelo agente de RL sejam executadas em tempo real nos processos de negócio.

1.2.2 Objetivos Específicos

- Levantar e comparar ferramentas de BPM open-source com suporte a exportação em formato estruturado, selecionando a mais adequada para representação de workflows como grafos direcionados.
- Definir e documentar três processos de negócio representativos com diferentes níveis de complexidade estrutural, que sirvam como base para treinamento, validação e comparação dos modelos de Machine Learning.
- Projetar e implementar um ambiente de simulação de workflows compatível com interfaces padrão de Reinforcement Learning, capaz de gerar estados, executar ações e calcular recompensas baseadas em métricas de complexidade e qualidade de processo.
- Desenvolver modelos de Graph Neural Networks para representar e extrair embeddings estruturais de workflows empresariais, capturando as relações topológicas entre nós (tarefas) e arestas (transições).

- Treinar e avaliar agentes de Reinforcement Learning para otimização de políticas de decisão em workflows, mensurando a redução de complexidade estrutural (número de nós e arestas) sem degradação das métricas de qualidade do processo.
- Projetar e implementar uma interface de integração via API REST entre o software BPM e o motor de Machine Learning, garantindo comunicação em tempo real, documentação automática e tempo de resposta dentro de parâmetros definidos.
- Gerar datasets sintéticos de execução de processos que simulem cenários reais incluindo casos normais, gargalos e exceções, validando a consistência estatística dos dados gerados.
- Comparar o desempenho dos agentes de RL com métodos baseline tradicionais de escalonamento de processos, quantificando o ganho obtido pela abordagem de Machine Learning em termos de throughput, tempo de ciclo e taxa de sucesso.
- Validar empiricamente a hipótese de que Reinforcement Learning combinado com Graph Neural Networks é capaz de otimizar workflows empresariais, reduzindo sua complexidade estrutural enquanto mantém ou melhora métricas de qualidade.

1.3 Abrangência

Para manter o foco e a viabilidade do projeto dentro do cronograma estabelecido, alguns elementos relevantes para implementações em ambiente de produção foram deliberadamente excluídos do escopo atual. Esta seção detalha cada um desses elementos, explicando sua importância no contexto geral de BPM e as razões para sua exclusão nesta fase. Estão dentro do escopo deste TCC todos os nove objetivos específicos enumerados na seção 1.2.2, abrangendo a seleção do motor BPM, a definição de processos representativos, a implementação do ambiente de simulação compatível com RL, o desenvolvimento dos modelos GNN, o treinamento e avaliação dos agentes, a integração via API REST, a geração de datasets sintéticos, a comparação com baselines tradicionais e a validação empírica da hipótese central.

Estão fora do escopo deste TCC, conforme listado a seguir, elementos que demandariam parcerias industriais e horizontes temporais incompatíveis com o cronograma estabelecido. Tais exclusões não invalidam a contribuição do trabalho, mas delimitam claramente seu caráter de prova de conceito técnica, abrindo caminho

para pesquisas subsequentes que possam abordar tais aspectos em contextos aplicados.

- Análise de custo financeiro (ROI, TCO)
- Integração com sistemas reais de produção
- Compliance e questões regulatórias
- Otimização de recursos humanos específicos

Quadro 1 – Escopo do projeto: dentro vs. fora

✓ DENTRO DO ESCOPO	✗ FORA DO ESCOPO
Otimização da estrutura do workflow (nós e arestas)	Análise de ROI e TCO
Métricas de tempo de ciclo e throughput	Integração com ERP, CRM, sistemas legados
Dados sintéticos simulando processos reais	Conformidade com LGPD, GDPR, normas setoriais
Recursos modelados como capacidades abstratas	Alocação de colaboradores individuais
Ambiente de simulação controlado	Deployment em ambiente de produção
Prova de conceito técnica	Explicabilidade de decisões (XAI)

Fonte: elaborado pelo autor (2026).

1.4 Proposta de Solução

Diante do problema enunciado na seção 1.1 — a ausência, nos motores de BPM atuais, de mecanismos automáticos capazes de reduzir a complexidade estrutural de workflows empresariais sem degradar suas métricas de qualidade —, este trabalho propõe uma solução que trata o processo de negócio como um grafo direcionado e aprende, por reforço, políticas de decisão que otimizam esse grafo durante a execução. A proposta articula três elementos. O primeiro é uma representação estrutural do workflow BPMN como grafo, processada por uma rede Graph Attention Network (GAT) que produz embeddings sensíveis à topologia. O segundo é um agente Proximal Policy Optimization (PPO) que, operando sobre esses embeddings, decide em cada ponto de ramificação a ação que maximiza uma função de recompensa multi-objetivo. O terceiro é a integração desse motor de decisão a um motor BPM real e open-source, o Camunda Platform 8, via interface REST de baixa latência.

A solução responde diretamente à questão geral de pesquisa — se técnicas de Machine Learning conseguem reduzir a complexidade estrutural de workflows sem perda de qualidade — e desdobra-se nas nove questões específicas formuladas na seção 2.2.1. Cada questão corresponde a um componente verificável da arquitetura: a escolha do motor BPM e do formato de intercâmbio (Q1), a definição de processos representativos (Q2), o ambiente de simulação (Q3), a arquitetura da rede em grafo (Q4), a capacidade do agente de reduzir complexidade preservando qualidade (Q5), o ganho quantitativo frente a baselines (Q6), a validade estatística dos datasets (Q7), os requisitos não-funcionais da integração (Q8) e a capacidade de generalização entre processos (Q9). O Quadro apresentado ao final da seção 2.2.1 sintetiza essa articulação, indicando em qual capítulo cada questão é endereçada.

Em síntese, a proposta de solução é detalhada arquiteturalmente no Capítulo 4, materializada no pipeline de referência descrito na seção 4.5, exercitada empiricamente no estudo de caso do Capítulo 5 e avaliada quantitativamente no Capítulo 6, em que cada uma das cinco proposições de pesquisa (P1 a P5) é confirmada ou refutada à luz das evidências coletadas. Desse modo, o encadeamento problema → questões → solução → evidências percorre o trabalho de ponta a ponta, conferindo-lhe rastreabilidade metodológica conforme o protocolo de estudo de caso de Yin (2018).

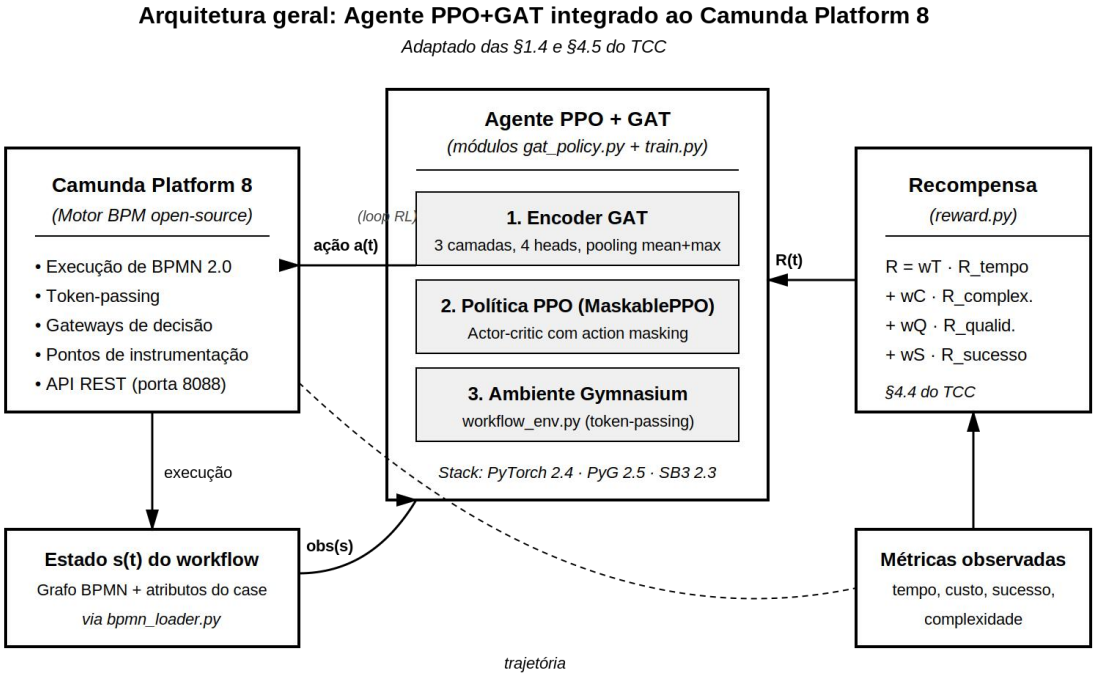


Figura 1 – Arquitetura geral do sistema proposto
Fonte: O autor (2026).

1.5 Organização do Trabalho

Este Trabalho de Conclusão de Curso está organizado em sete capítulos, além dos elementos pré-textuais (capa, folha de rosto, ficha catalográfica, dedicatória, agradecimentos, epígrafe, resumo, abstract, listas e sumário) e pós-textuais (glossário e referências bibliográficas).

O Capítulo 1 — Introdução apresenta o contexto geral do trabalho, contextualizando a pesquisa no cenário atual de Business Process Management e Inteligência Artificial. Inclui a motivação econômica, técnica e social, os objetivos geral e específicos, a abrangência (escopo) do trabalho e os principais trabalhos relacionados que sustentam a relevância da pesquisa.

O Capítulo 2 — Metodologia de Pesquisa descreve os critérios de seleção da metodologia adotada (Estudo de Caso) e detalha o projeto de pesquisa segundo os componentes propostos por Yin (2018): questões de estudo, proposições, unidade de análise, lógica que une os dados às proposições, critérios para interpretar as constatações, coleta de evidências e análise qualitativa.

O Capítulo 3 — Fundamentação Teórica da Pesquisa apresenta a revisão bibliográfica que sustenta o trabalho, abordando os fundamentos de Business Process Management, Reinforcement Learning (com ênfase em PPO), Graph Neural Networks (com ênfase em GAT), Process Mining e a integração de Machine Learning com BPM.

O Capítulo 4 — Aplicação Prática do Conceito da Pesquisa descreve a arquitetura proposta, o stack tecnológico selecionado, o pipeline GNN + Reinforcement Learning e a função de recompensa multi-objetivo, justificando todas as escolhas técnicas com base nos requisitos da pesquisa.

O Capítulo 5 — Estudo de Caso apresenta o caso original de 2018 (sistema de seleção e contratação de pessoal com lógica difusa) como referência histórica, e detalha os três processos representativos da proposta 2026 (seleção, compras e incidentes), bem como os datasets sintéticos gerados.

O Capítulo 6 — Avaliação dos Resultados define as métricas e hipóteses avaliadas, apresenta os resultados parciais obtidos durante o desenvolvimento, compara o desempenho do agente PPO + GAT com baselines tradicionais e contrasta as capacidades técnicas das versões 2018 e 2026.

O Capítulo 7 — Considerações Finais sintetiza as contribuições do trabalho, registra as recomendações para organizações interessadas em adotar a abordagem e indica trabalhos futuros que poderão estender a pesquisa em direções complementares.

Glossário e **Referências Bibliográficas** encerram o documento, fornecendo definições dos termos técnicos utilizados e a lista completa das obras citadas no formato ABNT NBR 6023:2018.

2 METODOLOGIA DE PESQUISA

Este capítulo apresenta a metodologia adotada para a condução desta pesquisa, descrevendo os critérios que justificaram a escolha do método de Estudo de Caso, bem como o detalhamento dos seus componentes essenciais segundo o referencial de Yin (2018): as questões de estudo, as proposições, a unidade de análise, a lógica que une os dados às proposições, os critérios para interpretar as constatações, a coleta de evidências e, por fim, a análise qualitativa das evidências.

2.1 Critérios de seleção da metodologia de pesquisa

Quanto à natureza, esta pesquisa caracteriza-se como aplicada, pois objetiva gerar conhecimento dirigido à solução de um problema concreto identificado em ambiente empresarial: a ausência de mecanismos automáticos de otimização estrutural em motores BPM. Quanto aos objetivos, classifica-se como exploratória e explicativa, pois investiga uma combinação tecnológica (GNN + RL aplicados a workflows BPMN) ainda pouco explorada na literatura e busca explicar empiricamente as relações de causa e efeito entre as escolhas arquiteturais propostas e a redução de complexidade dos processos.

Quanto à abordagem, predomina o método quantitativo, dado que a validação das hipóteses depende de métricas numéricas (número de nós, número de arestas, tempo de ciclo, throughput, taxa de sucesso, latência de inferência); contudo, observa-se também uma dimensão qualitativa na análise comparativa do stack tecnológico, na avaliação das limitações da abordagem original (2018) e na discussão dos trabalhos relacionados. Em relação aos procedimentos técnicos, a pesquisa combina revisão bibliográfica sistemática, pesquisa experimental sobre datasets sintéticos com características controladas, e pesquisa-ação na medida em que o autor participa ativamente do desenvolvimento, treinamento e avaliação dos artefatos computacionais resultantes.

A escolha do método de Estudo de Caso foi orientada por três critérios principais. Em primeiro lugar, segundo Yin (2018), o Estudo de Caso é especialmente adequado quando a questão de pesquisa pergunta “como” ou “por que” um determinado fenômeno contemporâneo ocorre dentro de um contexto da vida real, sobre o qual o pesquisador tem pouco ou nenhum controle. A questão central deste TCC — “é possível utilizar técnicas de Machine Learning para otimizar automaticamente a estrutura e execução de workflows empresariais?” — encaixa-se nessa caracterização.

Em segundo lugar, a unidade de análise selecionada (a integração entre o motor BPM Camunda Platform 8 e o motor de Machine Learning baseado em GAT + PPO) é um sistema sociotécnico complexo, cuja avaliação isolada por meio de experimentos puramente laboratoriais perderia parte do contexto operacional de processos de negócio reais. O Estudo de Caso permite observar o fenômeno de forma holística, integrando aspectos técnicos, arquiteturais e de modelagem de processos.

Em terceiro lugar, a possibilidade de instanciar o estudo em três processos de negócio representativos (seleção de pessoal, compras e incidentes), com graus distintos de complexidade estrutural, configura um Estudo de Caso múltiplo (multiple-case study), o que aumenta a robustez analítica da pesquisa em comparação a um caso único, permitindo testar a generalização das proposições entre cenários e identificar regularidades empíricas.

2.2 A Definição do Projeto de Pesquisa de Estudo de Caso

Conforme Yin (2018), o projeto de pesquisa de Estudo de Caso é estruturado em torno de cinco componentes principais — questões de estudo, proposições, unidade de análise, lógica que une os dados às proposições e critérios para interpretar as constatações — aos quais se acresce, neste trabalho, um sexto componente operacional: a estratégia de coleta de evidências. As próximas subseções detalham cada um desses componentes para o caso específico desta pesquisa.

2.2.1 As Questões de Estudo

As questões de estudo a seguir orientam a investigação e devem ser respondidas, no todo ou em parte, pela versão final deste TCC. Cada questão articula-se a um ou mais objetivos específicos enumerados na seção 1.2.2 e fornece a base para a definição das métricas de avaliação que serão consolidadas no Capítulo 6 (Avaliação dos Resultados):

- Q1 — Quais ferramentas BPM open-source disponíveis em 2026 oferecem suporte nativo à exportação de workflows em formato estruturado (preferencialmente BPMN 2.0) compatível com a representação grafo-teórica G(V, E) requerida pelo módulo de Graph Neural Networks?
- Q2 — Como definir e parametrizar processos de negócio representativos, com diferentes níveis de complexidade estrutural, de modo que sirvam de base reprodutível para o treinamento, validação e comparação dos modelos de Machine Learning propostos?
- Q3 — De que forma o ambiente de simulação compatível com a interface padrão do Gymnasium pode modelar fielmente a execução de workflows

BPMN, gerando estados, ações e recompensas que reflitam as métricas reais de complexidade e qualidade dos processos?

- Q4 — Qual arquitetura de Graph Neural Network (GAT, GCN, GraphSAGE ou variantes) produz embeddings mais informativos para representar a topologia dos workflows empresariais, e quais hiperparâmetros (número de camadas, dimensão dos embeddings, número de heads de atenção) maximizam essa informatividade?
- Q5 — Em que medida o algoritmo Proximal Policy Optimization (PPO), operando sobre embeddings produzidos por GNN, é capaz de aprender políticas de decisão que reduzam a complexidade estrutural dos workflows (número de nós e arestas) sem degradar as métricas de qualidade do processo (tempo de ciclo, throughput, taxa de sucesso)?
- Q6 — Qual é o ganho quantitativo (em termos percentuais) obtido pela abordagem GNN + RL em comparação a métodos baseline tradicionais de escalonamento de processos, e como esse ganho varia entre os três processos representativos definidos (seleção, compras, incidentes)?
- Q7 — Os datasets sintéticos gerados (6000 cases distribuídos em casos normais, gargalos e exceções) possuem consistência estatística e similaridade estrutural suficientes em relação a event logs reais publicamente disponíveis, validados por meio de técnicas de process discovery e conformance checking do PM4Py?
- Q8 — A interface de integração via API REST entre o software BPM (Camunda Platform 8) e o motor de Machine Learning atende aos requisitos de comunicação em tempo real, com latência de inferência inferior a 100 ms, documentação automática (OpenAPI) e robustez frente a falhas pontuais?
- Q9 — Em que grau um agente PPO + GNN treinado sobre um processo de negócio é capaz de generalizar suas políticas de decisão para outros processos com estrutura topológica similar, configurando uma forma efetiva de transfer learning entre domínios de workflow?

Quadro 2 – Articulação entre as questões de pesquisa, a proposta de solução e os capítulos que as endereçam

Questão	Foco	Como a proposta responde	Onde é endereçada
Q1	Motor BPM e formato BPMN 2.0	Camunda Platform 8 + parser BPMN→grafo	seção 4.2.1; seção 4.3.1
Q2	Processos representativos	Três processos de complexidade variada (A, B, C)	seção 5.2

Q3	Ambiente de simulação Gymnasium	Ambiente token-passing WorkflowEnv	seção 4.3.4; seção 5.4
Q4	Arquitetura da GNN	GAT/GATv2, 3 camadas, 4 heads de atenção	seção 3.3; seção 4.3.2
Q5	Reduzir complexidade sem perder qualidade	Agente PPO + recompensa multi-objetivo	seção 4.3.3; seção 4.4; seção 6.3.3
Q6	Ganho quantitativo vs. baselines	Comparação com FIFO e prioridade-fixa	seção 6.3.2; seção 6.3.3
Q7	Validade dos datasets sintéticos	Geração 70/20/10 + validação via PM4Py	seção 5.3; seção 6.3.1
Q8	Integração em tempo real (< 100 ms)	API FastAPI + teste de carga Locust	seção 4.5; seção 5.5.5; seção 6.3.4
Q9	Generalização entre processos	Experimento de transferência B → C	seção 5.5.4; seção 6.3.4

Fonte: O autor (2026).

2.2.2 As Proposições

Conforme Yin (2018), as proposições direcionam a atenção do pesquisador para aspectos relevantes do caso e antecipam relações de causa e efeito entre as variáveis investigadas, fornecendo as hipóteses operacionais a serem confirmadas ou refutadas pelas evidências coletadas. Para esta pesquisa, formulam-se cinco proposições centrais, derivadas da questão geral e articuladas com as questões de estudo Q4 a Q9:

- **P1 — Redução de complexidade estrutural:** um agente PPO operando sobre embeddings produzidos por uma rede GAT de três camadas e quatro heads de atenção é capaz de reduzir, em média, pelo menos 15% do número de nós e/ou arestas dos workflows otimizados, em relação à versão original definida pelo modelador humano.
- **P2 — Manutenção da qualidade do processo:** a redução de complexidade obtida pelo agente PPO + GAT não acarreta degradação superior a 5% nas métricas de qualidade do processo (tempo médio de ciclo, throughput agregado e taxa de sucesso), permanecendo dentro de uma faixa estatisticamente equivalente à versão original.
- **P3 — Latência de inferência viável em tempo real:** a inferência completa do pipeline GAT + PPO, executada via API REST FastAPI sobre os workflows representados como grafos direcionados, ocorre em tempo médio inferior a 100 ms por requisição, viabilizando a tomada de decisão em tempo real durante a execução dos processos no motor Camunda Platform 8.

- **P4 — Capacidade de generalização entre processos:** um agente PPO + GAT treinado sobre um dos três processos representativos (por exemplo, Compras) é capaz de obter desempenho ao menos 70% equivalente quando aplicado, sem retreinamento completo, a outro processo representativo com estrutura topológica similar (por exemplo, Incidentes), caracterizando uma forma efetiva de transfer learning.
- **P5 — Superioridade frente a baselines tradicionais:** o ganho quantitativo (em termos de redução de tempo de ciclo, aumento de throughput e taxa de sucesso) da abordagem GNN + RL é, em média, superior a 10% em relação a heurísticas tradicionais de escalonamento (FIFO, prioridade fixa) aplicadas aos mesmos cenários.

2.2.3 A Unidade de Análise

A unidade de análise deste estudo é o sistema integrado de otimização de workflows formado pela tríade Camunda Platform 8 (motor BPM) + Pipeline GAT + PPO (motor de inferência ML) + API REST FastAPI (camada de integração), aplicado a um conjunto de três processos de negócio representativos (seleção de pessoal, compras corporativas e gestão de incidentes), executado sobre 6.000 cases sintéticos distribuídos entre casos normais, gargalos e exceções.

A delimitação da unidade de análise foi orientada por três critérios. Primeiro, a unidade deve ser observável de forma integrada — ou seja, não basta avaliar isoladamente o desempenho da rede GAT ou do agente PPO; é necessário medir o sistema como um todo, em sua interação com o motor BPM. Segundo, a unidade deve ser instanciável em mais de um cenário, o que justifica a escolha de três processos com complexidade crescente, configurando um Estudo de Caso múltiplo. Terceiro, a unidade deve ser reprodutível, o que é assegurado pela utilização exclusiva de software open-source, datasets sintéticos com sementes determinísticas e código-fonte versionado.

São excluídos da unidade de análise, conforme a abrangência descrita na seção 1.3, aspectos relacionados a integração com sistemas reais de produção (ERP, CRM), análise financeira (ROI, TCO), conformidade regulatória (LGPD, GDPR), explicabilidade (XAI) e alocação individualizada de colaboradores. Estes pontos são reconhecidos como relevantes, mas configuram extensões para trabalhos futuros.

2.2.4 A Lógica que Une os Dados às Proposições

A lógica que une os dados coletados às proposições estabelecidas baseia-se em pattern matching (Yin, 2018), no qual padrões empíricos observados nos

experimentos são comparados com padrões previstos pelas proposições P1 a P5. Cada proposição é associada a um conjunto de métricas operacionais, definidas previamente, cuja medição em condições controladas permite confirmar ou refutar a hipótese correspondente.

Operacionalmente, esta lógica é instanciada da seguinte forma:

- **Para P1,** comparam-se o número de nós (|V|) e arestas (|E|) dos workflows antes e depois da otimização pelo agente PPO + GAT. A proposição é confirmada se a redução média for ≥ 15% para cada um dos três processos representativos.
- **Para P2,** medem-se as métricas de qualidade (tempo médio de ciclo, throughput, taxa de sucesso) antes e após a otimização, com testes estatísticos (t de Student pareado, com $\alpha = 0,05$) para verificar se a degradação se mantém dentro do limite de 5%.
- **Para P3,** registram-se os tempos de resposta (em milissegundos) de 1.000 requisições consecutivas à API REST sob carga controlada, calculando-se a média, mediana e percentil 95. A proposição é confirmada se o percentil 95 for inferior a 100 ms.
- **Para P4,** treina-se o agente em um processo (e.g., Compras) e avalia-se seu desempenho, sem retreinamento, em outro (e.g., Incidentes), comparando as métricas de redução de complexidade e qualidade. A proposição é confirmada se o desempenho relativo se mantiver acima de 70%.
- **Para P5,** comparam-se as métricas obtidas pelo agente PPO + GAT com as obtidas por baselines (FIFO, prioridade fixa) sobre os mesmos cases sintéticos, com testes estatísticos para confirmar a significância da diferença.

2.2.5 Os Critérios para Interpretar as Constatações

Os critérios para interpretar as constatações empíricas obedecem a três níveis: estatístico, técnico e conceitual.

Nível estatístico: são adotados testes paramétricos (t de Student pareado) para comparações entre médias, com nível de significância $\alpha = 0,05$. Em casos de violação dos pressupostos de normalidade (verificada por Shapiro-Wilk), recorre-se a alternativas não-paramétricas (Wilcoxon). Intervalos de confiança de 95% acompanham todas as métricas reportadas.

Nível técnico: considera-se viável tecnicamente uma solução cujo treinamento converge em até 10 milhões de timesteps em um servidor com GPU NVIDIA RTX

3060 ou equivalente, e cuja inferência satisfaz as restrições de latência (< 100 ms p95) descritas na proposição P3.

Nível conceitual: uma proposição é considerada confirmada quando, simultaneamente, (i) o padrão observado coincide com o padrão previsto, (ii) a confirmação se mantém em pelo menos dois dos três processos representativos, e (iii) os resultados são reproduzíveis a partir do código e dos datasets disponibilizados publicamente. Reciprocamente, uma proposição é refutada se as evidências contradizem o padrão previsto em pelo menos dois dos três processos.

Em situações intermediárias, em que apenas um dos três processos confirma a proposição, classifica-se o resultado como “parcialmente confirmado” e discute-se, no Capítulo 6, as razões prováveis da divergência (e.g., topologia atípica, distribuição de tarefas, ruído nos dados sintéticos).

2.2.6 A Coleta de Evidências

A coleta de evidências adota múltiplas fontes, conforme recomendado por Yin (2018) para fortalecer a validade do estudo por meio de triangulação. As fontes utilizadas são:

- **Documentação técnica:** manuais oficiais de Camunda Platform 8, PyTorch, PyTorch Geometric, Stable-Baselines3, FastAPI e PM4Py; especificação BPMN 2.0 da OMG; documentação ABNT (NBR 6023, NBR 10520, NBR 14724).
- **Literatura científica:** artigos publicados em conferências e periódicos indexados (Qualis A1–B2), com ênfase nos trabalhos identificados na seção 1.4 (Huang et al., 2011; Silvander, 2019; Soliman, Mostafa e Younis, 2024) e em livros-referência (Sutton; Barto, 2018; Hamilton, 2020).
- **Logs de execução:** event logs sintéticos gerados via Faker e PM4Py, no formato XES, com 6.000 cases distribuídos entre os três processos representativos.
- **Métricas de treinamento:** séries temporais de recompensa, perda da política, perda da função de valor, divergência KL e advantage, registradas via MLflow e Weights & Biases ao longo do treinamento do agente PPO.
- **Métricas de inferência:** tempos de resposta da API REST FastAPI, throughput de inferência (req/s), uso de CPU, GPU e memória, registrados via Prometheus e visualizados via Grafana.

- **Métricas de processo:** número de nós e arestas dos workflows otimizados, tempo de ciclo, throughput agregado, taxa de sucesso, calculados a partir dos event logs no PM4Py.

2.3 A Análise Qualitativa das Evidências

Embora a maior parte das evidências coletadas seja de natureza quantitativa, a análise qualitativa cumpre papel essencial na pesquisa, especialmente nas comparações de stack tecnológico, na avaliação das limitações da abordagem original (2018) e na interpretação de comportamentos não previstos do agente. Nesta seção, descrevem-se as estratégias de análise qualitativa adotadas.

A primeira estratégia é a análise comparativa estruturada (cross-case synthesis), aplicada às tabelas de comparação 2018 vs. 2026 (Tabelas 2 e 3 do trabalho) e à comparação entre motores BPM (Tabela 1). Para cada critério, atribui-se uma classificação qualitativa (Completo, Parcial, Limitado, Ausente) que permite visualizar diferenças relativas entre as alternativas avaliadas.

A segunda estratégia é a análise de trajetória (process tracing), aplicada à inspeção dos workflows otimizados pelo agente PPO. Para cada workflow, registra-se a sequência de decisões tomadas, os estados intermediários e as recompensas associadas, buscando identificar padrões recorrentes de otimização (e.g., paralelização de tarefas independentes, eliminação de gateways redundantes, reordenação de aprovações).

A terceira estratégia é a análise de discrepâncias (deviant case analysis), focada nos casos em que o agente apresenta desempenho atípico — seja excepcionalmente bom, seja excepcionalmente ruim. Estes casos são examinados manualmente para identificar características topológicas particulares (e.g., presença de loops, alta densidade de gateways exclusivos) que possam explicar o comportamento e orientar ajustes na função de recompensa ou na arquitetura da rede.

Por fim, a quarta estratégia é a triangulação metodológica entre fontes (documentação, literatura, logs, métricas e inspeção manual), buscando convergência ou divergência entre as evidências. Quando as fontes convergem, a constatação é considerada robusta; quando divergem, registra-se a divergência e investigam-se suas causas, o que enriquece a discussão dos resultados no Capítulo 6.

2.4 Procedimentos de Revisão Sistemática da Literatura

O levantamento bibliográfico que fundamenta o Capítulo 3 e a discussão dos trabalhos correlatos foi conduzido de acordo com diretrizes consolidadas de revisão sistemática da literatura, combinando o protocolo PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) para reporte e seleção de evidências (Page et al., 2021) e a metodologia de Revisão Sistemática para Engenharia de Software de Kitchenham e Charters (2007). A adoção desses procedimentos visa garantir transparência, reprodutibilidade e auditabilidade das fontes utilizadas, em alinhamento com o caráter empírico do estudo de caso de Yin (2018) adotado nesta pesquisa.

2.4.1 Planejamento da Revisão (Kitchenham)

Seguindo Kitchenham e Charters (2007), o planejamento da revisão foi estruturado em três fases: (i) identificação da necessidade da revisão, motivada pela escassez de aplicações práticas de Reinforcement Learning e Graph Neural Networks em motores BPM cloud-native; (ii) formulação das questões de pesquisa, alinhadas às hipóteses do estudo (redução de complexidade estrutural, manutenção de qualidade e capacidade de generalização); e (iii) definição do protocolo de revisão, incluindo critérios de inclusão e exclusão, estratégia de busca e procedimentos de extração e síntese de dados.

As bases consultadas foram IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink, Scopus e Web of Science. A string de busca combinou os blocos conceituais centrais da pesquisa por meio dos operadores booleanos AND e OR, contemplando os termos: ("Business Process Management" OR "BPM" OR "workflow") AND ("Reinforcement Learning" OR "Deep Reinforcement Learning") AND ("Graph Neural Network" OR "Graph Attention" OR "GNN"). A janela temporal adotada foi 2018-2025, justificada pelo surgimento de arquiteturas GNN modernas (GAT em 2018) e pela publicação do PPO em 2017 como marco do RL contemporâneo.

2.4.2 Condução e Reporte segundo PRISMA 2020

A condução da revisão seguiu o fluxo PRISMA 2020 (Page et al., 2021), organizado em quatro etapas: (1) Identificação, registrando o número total de estudos recuperados por base; (2) Triagem, com remoção de duplicatas e análise de títulos e resumos; (3) Elegibilidade, com leitura integral dos textos remanescentes e aplicação dos critérios de inclusão (apresentação de método experimental, integração explícita entre BPM e Machine Learning, métricas quantitativas reportadas) e exclusão (estudos puramente teóricos sem validação empírica, trabalhos sem peer review ou anteriores a 2018); e (4) Inclusão, consolidando o corpus final efetivamente analisado no Capítulo 3.

2.4.3 Articulação com o Estudo de Caso de Yin

A revisão sistemática descrita acima cumpre dois papéis complementares dentro da arquitetura metodológica de Yin (2018) adotada nesta pesquisa. No nível da fundamentação teórica (Capítulo 3), o corpus selecionado oferece a base conceitual e o estado da arte para os blocos teóricos de BPM, Reinforcement Learning, Graph Neural Networks e Process Mining. No nível do estudo de caso (Capítulos 4 e 5), os trabalhos elegíveis funcionam como referencial comparativo para a triangulação

metodológica e para a interpretação das constatações empíricas, conforme detalhado no Capítulo 6.

2.4.4 Síntese das Lacunas Identificadas na Literatura

Esta subseção sintetiza os resultados da revisão exploratória conduzida segundo o protocolo descrito nas subseções 2.4.1 e 2.4.2. As buscas executadas nas seis bases bibliográficas indicadas (IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink, Scopus e Web of Science), restritas ao período 2018-2025 — com exceção pontual de obras seminais anteriores —, produziram um corpus de trabalhos cuja análise temática convergiu para três lacunas centrais. Tais lacunas fundamentam as escolhas arquiteturais e metodológicas detalhadas nos Capítulos 4 e 5, sustentando o encadeamento problema → questões → solução enunciado na seção 1.4.

Primeira lacuna — automação da decisão estrutural em motores BPM. Refere-se à ausência, nos motores BPM correntes — sejam comerciais ou de código aberto —, de mecanismos automáticos de decisão sobre a própria estrutura do workflow durante a execução. Embora a literatura reconheça o BPM como pilar operacional das organizações modernas (Hitpass, 2017) e identifique a inteligência artificial como vetor de transformação dessa tecnologia (Qian, 2017), o estado da arte revela que a tomada de decisão automatizada sobre fluxos permanece pouco endereçada pelas plataformas em produção. A revisão conduzida por Afifi, Khebizi e Halimi (2024) sobre abordagens de process mining confirma esse diagnóstico, apontando que sistemas adaptativos capazes de aprender com o histórico operacional e otimizar dinamicamente o fluxo permanecem objeto de pesquisa, sem internalização nos produtos correntes.

Segunda lacuna — aplicação de Reinforcement Learning a process mining e BPM. Concerne ao caráter de fronteira da aplicação de RL a problemas de mineração e otimização de processos, ainda sem soluções consolidadas em ambiente industrial. O trabalho seminal de Huang, van der Aalst, Lu e Duan (2011), publicado em Data & Knowledge Engineering, demonstrou empiricamente a viabilidade de RL para alocação de recursos em BPM, abrindo a frente de pesquisa expandida por Silvander (2019) — com foco em otimização de processos via RL — e por Soliman, Mostafa e Younis (2024), com aplicação específica a mineração e otimização de processos. A revisão sistemática de Silva e Marreiros (2024) sobre mineração preditiva de processos e o levantamento de Dogru et al. (2024) sobre RL em processos industriais — domínio adjacente — confirmam o caráter incipiente da consolidação industrial. Não foram identificadas, no corpus analisado, arquiteturas que integrem nativamente

um agente RL a um motor BPM de produção, tampouco que combinem RL com representação grafo-teórica do workflow.

Terceira lacuna — representação topológica de workflows em modelos de Machine Learning. Diz respeito à representação computacional adotada pela maior parte dos trabalhos identificados: empregam-se predominantemente representações tabulares (atributos planos por case) ou sequenciais (event logs como séries temporais), o que descarta a riqueza topológica intrínseca aos workflows — paralelismos, ciclos, dependências de longo alcance (Afifi; Khebizi; Halimi, 2024). Os avanços recentes em Graph Neural Networks, particularmente Graph Attention Networks (Veličković et al., 2018) e sua variante GATv2 (Brody; Alon; Yahav, 2022), oferecem o instrumental necessário para superar essa limitação, conforme sistematizado por Hamilton (2020). Posições recentes como a de Zhang et al. (2025), que formaliza workflows como grafos computacionais para inferência via GNN, validam a pertinência da abordagem grafo-teórica para inferir propriedades de desempenho de workflows; entretanto, sua aplicação específica a Business Process Management permanece pouco explorada, configurando o espaço de contribuição visado por esta pesquisa.

As três lacunas, lidas conjuntamente, fundamentam as escolhas centrais desta pesquisa e respondem antecipadamente, em nível de motivação, às nove questões formuladas na seção 2.2.1. A integração da solução a um motor BPM real e open-source — o Camunda Platform 8 — responde à primeira lacuna (e às questões Q1 e Q8); a adoção do PPO (Schulman et al., 2017) como núcleo de decisão, combinada à comparação sistemática contra baselines tradicionais, responde à segunda lacuna (e às questões Q5, Q6 e Q9); a opção por GAT processando a representação grafo-teórica do BPMN responde à terceira lacuna (e às questões Q3, Q4 e Q7). A execução dessa síntese é detalhada nos Capítulos 4, 5 e 6, com a ressalva de que os resultados quantitativos reportados no Capítulo 6 são preliminares, decorrentes do estudo piloto descrito em 5.4.

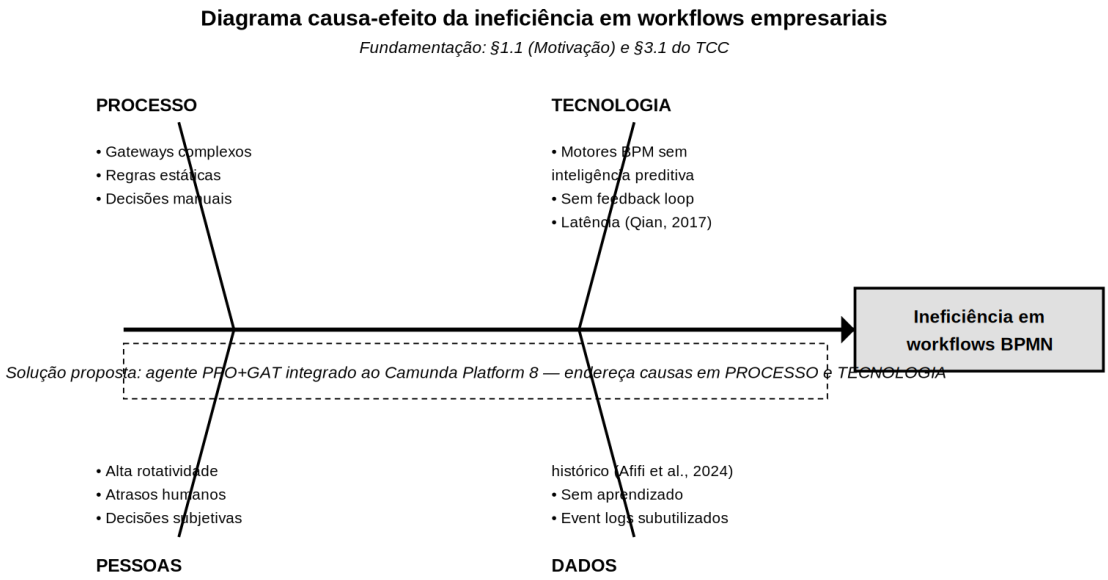


Figura 5 – Diagrama causa-efeito da ineficiência em workflows empresariais
Fonte: O autor (2026).

3 FUNDAMENTAÇÃO TEÓRICA DA PESQUISA

Este capítulo apresenta a fundamentação teórica que sustenta a proposta de integração entre Graph Neural Networks e Reinforcement Learning para otimização de workflows empresariais. Diferentemente de abordagens superficiais que apenas aplicam ferramentas existentes, propõe-se uma arquitetura original que combina representações vetoriais de grafos com aprendizado por reforço para resolver um problema até então não endereçado adequadamente. As seções a seguir cobrem, em ordem, os fundamentos de Business Process Management (3.1), Reinforcement Learning (3.2), Graph Neural Networks (3.3), Process Mining (3.4) e a integração de Machine Learning com BPM (3.5).

3.1 Business Process Management (BPM)

Business Process Management (BPM) é uma disciplina que combina conhecimentos de Tecnologia da Informação e ciências gerenciais com o objetivo de modelar, automatizar, executar, controlar, mensurar e otimizar fluxos de atividades empresariais — denominados processos de negócio — em apoio aos objetivos estratégicos da organização (Hitpass, 2017). Diferentemente de abordagens funcionais clássicas, BPM enfatiza a visão horizontal do trabalho: processos atravessam departamentos e sistemas, e seu desempenho depende da coordenação eficaz entre tarefas humanas e automatizadas.

Um processo de negócio é, formalmente, um conjunto ordenado de atividades executadas para entregar um produto ou serviço a um cliente, interno ou externo. Hitpass (2017) classifica os processos em três categorias: estratégicos (que orientam decisões de longo prazo), primários ou de cadeia de valor (que produzem diretamente o produto entregue ao cliente) e de suporte (que sustentam a operação dos demais). A maturidade BPM de uma organização é avaliada pela capacidade de gerenciar e otimizar continuamente esses processos.

3.1.1 Conceitos Fundamentais

Os conceitos fundamentais que sustentam a prática BPM incluem: instância de processo (case), tarefa (task), recurso (resource), gateway (ponto de decisão), evento (event) e fluxo de sequência (sequence flow). Cada instância representa uma execução particular do processo — por exemplo, a contratação de um candidato específico — e percorre uma sequência de tarefas, decidida em tempo de execução pelos gateways e pelos eventos disparados por sistemas externos ou ações humanas.

Um aspecto central, e particularmente relevante para esta pesquisa, é a distinção entre o modelo do processo (process model) e a execução do processo (process instance). O modelo descreve o comportamento esperado do processo em forma de diagrama, enquanto a execução produz event logs que documentam o que efetivamente ocorreu em cada instância. Esta dicotomia entre o prescrito e o realizado é a raiz da disciplina de Process Mining (seção 3.4) e abre espaço para abordagens de otimização baseadas em dados, como a proposta neste TCC.

3.1.2 BPMN 2.0 e Representação de Workflows

Business Process Model and Notation (BPMN) é um padrão internacional, mantido pela Object Management Group (OMG), que define uma notação gráfica para modelagem de processos de negócio. A versão 2.0, publicada em 2011 e amplamente adotada na indústria, define quatro categorias principais de elementos: objetos de fluxo (atividades, eventos, gateways), conectores (fluxos de sequência, fluxos de mensagem, associações), swimlanes (pools e lanes) e artefatos (anotações, grupos, objetos de dados). A padronização de fluxos de trabalho e da interoperabilidade entre sistemas de workflow remonta aos modelos de referência estabelecidos pela Workflow Management Coalition (WfMC, 2018).

A virtude do BPMN 2.0 é fornecer uma representação simultaneamente legível por humanos e processável por máquinas. Cada diagrama BPMN possui um arquivo XML correspondente que pode ser interpretado por motores de execução (e.g., Camunda Platform 8) e convertido em estruturas de dados algorítmicas. Para os propósitos deste TCC, esta dualidade é crucial: o modelo BPMN é convertido em um

grafo direcionado G(V, E), onde V representa o conjunto de tarefas, eventos e gateways, e E representa o conjunto de fluxos de sequência. Esta representação grafo-teórica é a entrada da Graph Neural Network responsável por gerar embeddings.

Adicionalmente, BPMN 2.0 suporta a anotação de atributos não-funcionais (duração esperada, custo, recurso responsável, prioridade), que podem ser codificados como features de nó na representação grafo-teórica. Esta riqueza semântica diferencia BPMN de notações puramente sintáticas, como DAGs do Apache Airflow, e justifica sua escolha como padrão de modelagem neste trabalho.

3.1.3 Ciclo de Vida BPM

O ciclo de vida BPM, conforme proposto por Hitpass (2017) e amplamente adotado na literatura, é composto por seis fases iterativas: planejamento estratégico, desenho/modelagem, implementação, execução, monitoramento e otimização. A fase de otimização — foco deste TCC — depende crucialmente das fases anteriores: somente um processo bem modelado, executado e monitorado fornece o volume e a qualidade de event logs necessários para alimentar técnicas de aprendizado de máquina.

A automação de cada uma destas fases tem evoluído rapidamente. A modelagem é apoiada por ferramentas visuais (Camunda Modeler); a execução, por motores cloud-native (Zeebe); o monitoramento, por dashboards em tempo real (Camunda Operate); e a otimização, historicamente, por análises ad hoc realizadas por especialistas. A presente pesquisa propõe que esta última fase — a otimização — seja também automatizada, por meio do pipeline GNN + RL descrito nos capítulos seguintes.

3.2 Reinforcement Learning

Reinforcement Learning (RL), ou aprendizado por reforço, é o paradigma de Machine Learning no qual um agente aprende a tomar decisões sequenciais através da interação com um ambiente, recebendo recompensas ou penalidades como retorno de suas ações (Sutton; Barto, 2018). Diferentemente do aprendizado supervisionado, o RL não dispõe de um conjunto rotulado de exemplos corretos; o agente deve descobrir, por tentativa e erro, quais ações maximizam o retorno acumulado ao longo do tempo. Esta característica torna o RL particularmente adequado para problemas em que a relação entre ação e recompensa é indireta, retardada ou não-determinística — exatamente o caso da otimização de workflows.

No contexto de BPM, o agente de RL pode ser visto como um “modelador autônomo” que, em cada ponto de decisão de um workflow, escolhe a ação que maximiza uma combinação ponderada de objetivos — tempo de ciclo, throughput, taxa de sucesso. Esta visão alinha-se com os trabalhos de Huang et al. (2011), Silvander (2019) e Soliman, Mostafa e Younis (2024), que pavimentaram o caminho da aplicação de RL em problemas de processos.

3.2.1 Markov Decision Process (MDP)

O framework matemático fundamental que sustenta o Reinforcement Learning é o Markov Decision Process (MDP), formalizado pela tupla (S, A, P, R, γ), onde: S é o conjunto de estados possíveis do ambiente; A é o conjunto de ações disponíveis ao agente; P(s' | s, a) é a função de transição que descreve a probabilidade de chegar ao estado s' a partir de s ao executar a ação a; R(s, a) é a função de recompensa, que retorna um escalar associado à execução de a em s; e $\gamma \in [0, 1]$ é o fator de desconto, que pondera recompensas futuras em relação às imediatas (Sutton; Barto, 2018).

A propriedade de Markov, central ao MDP, postula que o estado seguinte depende apenas do estado atual e da ação tomada — não de toda a história anterior. Em workflows BPMN, esta propriedade é satisfeita ao se incluir, na representação de estado, todos os elementos relevantes: instâncias ativas, recursos disponíveis, métricas correntes do processo. Quando o estado é parcialmente observável, recorre-se a variantes (Partially Observable MDP — POMDP), com tratamento via redes recorrentes ou janelas históricas.

O objetivo do agente é encontrar uma política π(a | s) — uma distribuição de probabilidade sobre ações condicionada a estados — que maximize o retorno esperado descontado $E[\Sigma\ \gamma^t\ R(s_t, a_t)]$. Sutton e Barto (2018) demonstram que, para qualquer MDP finito, existe pelo menos uma política ótima π*, e que esta pode ser obtida iterativamente via algoritmos baseados em valor (Q-Learning, DQN) ou em política (Policy Gradient, PPO).

3.2.2 Família de Algoritmos: Value-based, Policy-based e Actor-Critic

Os algoritmos de Reinforcement Learning podem ser classificados em três grandes famílias. Os algoritmos baseados em valor (value-based), como Q-Learning e DQN, estimam a função de valor Q(s, a) — o retorno esperado ao tomar a ação a no estado s seguindo a política ótima — e derivam a política implicitamente, escolhendo, em cada estado, a ação de maior Q-valor. Esta classe é eficiente em ambientes com espaços de ação discretos e pequenos, mas escala mal quando o número de ações cresce ou quando as ações são contínuas.

Os algoritmos baseados em política (policy-based), como REINFORCE e Policy Gradient, otimizam diretamente os parâmetros θ da política π_θ via gradiente, sem necessidade de estimar Q(s, a) explicitamente. Esta abordagem é naturalmente adequada a espaços de ação contínuos e a políticas estocásticas, e possui melhor garantia de convergência, embora frequentemente sofra de alta variância no estimador do gradiente.

Os algoritmos actor-critic combinam as duas abordagens: um “ator” (actor) representa a política, enquanto um “crítico” (critic) estima a função de valor para reduzir a variância do gradiente. A2C, A3C, PPO e SAC são exemplos representativos desta família. PPO, em particular, foi escolhido para esta pesquisa por sua estabilidade superior, conforme detalhado a seguir.

3.2.3 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) é um algoritmo de policy gradient proposto por Schulman et al. (2017) que se tornou um dos algoritmos de RL mais utilizados na prática moderna, sendo a base, por exemplo, do treinamento de Large Language Models via RLHF (Reinforcement Learning from Human Feedback). A inovação central do PPO está em seu mecanismo de clipping, que limita a magnitude das atualizações de política a cada step de treinamento, prevenindo mudanças abruptas que poderiam desestabilizar o aprendizado.

Formalmente, PPO maximiza uma função objetivo substituta (surrogate objective) da forma:

$$L^{CLIP}(\theta) = E_t[\, \min(r_t(\theta)\, \hat{A}_t ,\ clip(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\, \hat{A}_t) \,]$$

onde $r_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{\theta_{old}}(a_t \mid s_t)$ é a razão entre as políticas nova e antiga, $\hat{A}_t$ é a estimativa de advantage no tempo t, e ε é um hiperparâmetro tipicamente fixado em 0,2. O operador clip(·, 1-ε, 1+ε) impede que r_t se desvie excessivamente de 1, controlando o tamanho da atualização.

As vantagens práticas do PPO, exaustivamente reportadas na literatura, incluem: (i) estabilidade de treinamento superior a algoritmos como TRPO e A2C, (ii) eficiência amostral compatível com grandes batches de experiência coletada paralelamente, (iii) implementação relativamente simples, sem necessidade de aproximação de segunda ordem como TRPO, e (iv) bom desempenho em uma ampla variedade de domínios — de jogos Atari a robótica, de navegação até alinhamento de modelos de linguagem.

No contexto desta pesquisa, PPO é particularmente adequado por três razões. Primeiro, o espaço de ações em otimização de workflows pode ser grande (escolha

entre múltiplos caminhos paralelos, alocação entre dezenas de recursos), o que torna policy gradient mais escalável que value-based. Segundo, a exigência de baixa variância e estabilidade no treinamento é crítica em aplicações empresariais, em que flutuações severas no comportamento do agente seriam inaceitáveis. Terceiro, a disponibilidade de implementações maduras (Stable-Baselines3, RLlib) reduz drasticamente o esforço de desenvolvimento e aumenta a reprodutibilidade dos resultados.

Os hiperparâmetros iniciais utilizados nesta pesquisa seguem as recomendações da literatura: learning rate de 3e-4, fator de desconto $\gamma = 0{,}99$, parâmetro GAE $\lambda = 0{,}95$, ε de clipping = 0,2, número de epochs por batch = 10, batch size = 64 e horizon de coleta de 2.048 timesteps. Estes valores serão posteriormente refinados via grid search e Bayesian optimization, com tracking de experimentos em MLflow.

3.2.4 Aplicações de RL em Process Mining

A literatura recente registra uma fronteira ativa de pesquisa na aplicação de Reinforcement Learning a problemas de Process Mining e BPM. Soliman, Mostafa e Younis (2024) propõem um sistema baseado em RL para a identificação automática de sequências ótimas de transições em processos de negócio, partindo de event logs existentes. Os autores destacam que a complexidade combinatorial das possíveis sequências torna a busca exaustiva inviável para processos reais, justificando a abordagem aproximativa via RL.

Dogru et al. (2024), em revisão recente publicada no IEEE/CAA Journal of Automatica Sinica, sistematizam o estado da arte de RL aplicado a indústrias de processo (process industries), categorizando aplicações em três famílias: controle (process control), otimização (process optimization) e tomada de decisão (decision-making). Os autores identificam a otimização de workflows como uma área promissora, ainda sub-explorada, para aplicação industrial de RL.

Silvander (2019), em trabalho pioneiro neste domínio, demonstrou a viabilidade de Deep RL para parametrização de processos em sistemas de suporte a negócios, focando especificamente nos desafios de configuração e estabilidade. Embora não tenha utilizado representações grafo-teóricas — limitando-se a vetores de features tabulares —, o trabalho estabeleceu a viabilidade prática do paradigma e influenciou diretamente a arquitetura proposta neste TCC.

3.3 Graph Neural Networks

Graph Neural Networks (GNNs) constituem uma classe de redes neurais projetada para processar dados estruturados como grafos, generalizando arquiteturas como Convolutional Neural Networks (CNNs) e Recurrent Neural Networks (RNNs) para topologias não-euclidianas (Hamilton, 2020). Sua relevância crescente reflete o reconhecimento de que muitos problemas reais — redes sociais, moléculas químicas, redes de transporte, e, no caso desta pesquisa, workflows BPMN — possuem estrutura intrinsecamente grafo-teórica que seria perdida ao serem achatados em vetores.

3.3.1 Conceitos Fundamentais e Message Passing

Formalmente, um grafo é uma tupla G = (V, E), onde V é o conjunto de nós (vertices) e E ⊆ V × V é o conjunto de arestas (edges). Cada nó v ∈ V pode ser associado a um vetor de features x_v ∈ ℝ^d, e cada aresta (u, v) ∈ E pode opcionalmente possuir features adicionais. Em workflows BPMN, os nós representam tarefas, eventos e gateways, enquanto as arestas representam fluxos de sequência.

O paradigma central das GNNs é o message passing, formulado por Gilmer et al. (2017) e generalizado por Hamilton (2020). Em cada camada da rede, cada nó atualiza sua representação combinando: (i) sua própria representação anterior, e (ii) mensagens agregadas de seus vizinhos. Formalmente:

$$h_v^{(l+1)} = UPDATE^{(l)} (h_v^{(l)} , AGGREGATE^{(l)} (\{ h_u^{(l)} : u \in N(v) \}))$$

onde h_v^(l) é a representação do nó v na camada l, N(v) é a vizinhança (eventualmente direcional) de v, e UPDATE e AGGREGATE são funções diferenciáveis. Após L camadas de message passing, cada nó possui uma representação contextualizada que incorpora informação de seus vizinhos até L saltos de distância. Esta propriedade é fundamental para capturar dependências de médio e longo alcance em workflows.

As três principais arquiteturas de GNN diferem essencialmente nas funções AGGREGATE e UPDATE. Em GCN (Graph Convolutional Network), AGGREGATE é uma média ponderada pelo grau dos nós (Kipf; Welling, 2017). Em GraphSAGE, AGGREGATE pode ser média, max-pooling ou LSTM, e o processamento é feito sobre amostras da vizinhança para escalar a grafos grandes (Hamilton; Ying; Leskovec, 2017). Em GAT (Graph Attention Network), AGGREGATE utiliza pesos de atenção aprendidos, permitindo ao modelo focar nos vizinhos mais relevantes (Veličković et al., 2018).

3.3.2 Graph Attention Networks (GAT)

Graph Attention Network (GAT), proposta por Veličković et al. (2018), foi a arquitetura selecionada para esta pesquisa. A inovação central do GAT é o uso de mecanismo de atenção self-attention sobre os vizinhos de cada nó, calculando coeficientes de atenção α_{uv} que indicam a importância relativa de cada vizinho u para o nó central v:

$$\alpha_{uv} = softmax_u (LeakyReLU(a^T [W h_v \| W h_u]))$$

onde W é uma matriz de pesos compartilhada, a é um vetor de pesos da atenção, ‖ denota concatenação, e o softmax é normalizado sobre os vizinhos do nó v. A representação atualizada de v é então a soma ponderada das representações dos vizinhos: h_v^(l+1) = σ(Σ_u α_{uv} W h_u^(l)).

Para aumentar a estabilidade e a capacidade representacional, GAT utiliza multi-head attention: K cabeças de atenção independentes calculam, em paralelo, suas próprias representações, que são posteriormente concatenadas (em camadas intermediárias) ou agregadas por média (na camada final). Nesta pesquisa, K = 4 cabeças foi escolhido, equilibrando capacidade e custo computacional.

As vantagens do GAT, particularmente relevantes para workflows BPMN, são três. Primeiro, o mecanismo de atenção permite ao modelo aprender que algumas conexões entre tarefas são críticas (e.g., dependências obrigatórias) enquanto outras são acessórias (e.g., caminhos alternativos raramente percorridos). Segundo, GAT lida naturalmente com grafos de tamanho variável, sem necessidade de batch padding artificial. Terceiro, GAT é robusto a ruído estrutural (e.g., arestas espúrias provenientes de modelagem imprecisa), pois pode atribuir baixa atenção a vizinhos pouco informativos.

A configuração utilizada nesta pesquisa consiste em 3 camadas GAT com 4 heads de atenção cada, dimensão de embedding intermediária de 64 e ativação ELU entre camadas. A escolha de 3 camadas permite agregação de informação de até 3 saltos, suficiente para capturar a maioria das dependências em workflows típicos. Configurações com mais camadas resultam em over-smoothing — fenômeno em que as representações dos nós convergem para vetores indistinguíveis (Hamilton, 2020) —, fenômeno verificado em experimentos preliminares.

3.3.3 Embeddings e Pooling

O resultado final da rede GAT é, para cada nó v do workflow, um embedding contextualizado h_v ∈ ℝ^64. Para obter uma representação única do workflow inteiro — necessária para alimentar o agente PPO — aplica-se uma operação de pooling

global. Três opções foram consideradas: global mean pooling (média dos embeddings dos nós), global max pooling (máximo elemento a elemento) e global add pooling (soma).

Optou-se pela combinação mean + max pooling, que concatena os dois resultados, produzindo um vetor de dimensão 128. Esta abordagem captura simultaneamente o comportamento médio do workflow (via mean) e suas características extremas (via max), oferecendo representação mais rica que qualquer pooling individual. Trabalhos recentes em classificação de grafos confirmam essa escolha como robusta empiricamente.

O embedding global resultante serve como representação de estado para o agente PPO. Esta arquitetura end-to-end (GAT → pooling → PPO) permite treinamento conjunto: a GAT aprende não apenas representações genericamente boas do grafo, mas representações especificamente úteis para a tarefa de otimização. Esta propriedade — task-aware representation learning — é um diferencial importante em relação a abordagens que utilizam embeddings pré-treinados de forma desacoplada.

3.3.4 Aplicações de GNN em Sistemas Estruturais

As GNNs têm sido aplicadas com sucesso em diversos domínios estruturais. Zheng, Yu e Chen (2024), em trabalho recente publicado em Neural Networks, propõem Ci-GNN, uma rede neural baseada em causalidade de Granger para diagnóstico psiquiátrico interpretável a partir de redes cerebrais. O trabalho ilustra a versatilidade do paradigma GNN para domínios em que tanto a topologia quanto a interpretabilidade são críticas.

Em domínio correlato, Zhang et al. (2025), em artigo de posição apresentado no LoG 2025, formalizam workflows agênticos de modelos de linguagem como grafos computacionais e demonstram que redes neurais em grafos são preditores eficientes do desempenho desses workflows. Embora o domínio dos autores — workflows de múltiplos agentes de linguagem — difira dos processos de negócio BPMN tratados nesta pesquisa, a ideia central é convergente: representar workflows como grafos e processá-los com GNNs para inferir propriedades de desempenho, o que reforça a pertinência da abordagem grafo-teórica adotada neste trabalho.

Para o domínio específico de workflows e processos de negócio, a aplicação de GNNs ainda é incipiente. Afifi, Khebizi e Halimi (2024), em revisão sistemática publicada no International Journal of Business Process Integration and Management, identificam que a maioria dos trabalhos de Process Mining baseado em ML ainda

utiliza representações tabulares ou sequenciais, ignorando a riqueza topológica dos processos. Esta lacuna é exatamente a oportunidade explorada por este TCC.

3.4 Process Mining

Process Mining é a disciplina que aplica técnicas de mineração de dados a event logs gerados por sistemas de informação para descobrir, monitorar e melhorar processos reais. Como destacado por Silva e Marreiros (2024) em revisão sistemática recente, Process Mining preditivo (Predictive Process Mining) tem crescido em importância como ponte entre BPM e Machine Learning, oferecendo capacidades de antecipação que complementam as análises retrospectivas tradicionais.

As três tarefas clássicas de Process Mining são: (i) descoberta de processos (process discovery), em que um modelo é inferido a partir dos event logs; (ii) verificação de conformidade (conformance checking), em que se compara o que ocorreu com o que estava prescrito no modelo; e (iii) melhoria de processos (process enhancement), em que o modelo é estendido ou refinado com informações dos logs. A presente pesquisa contribui especialmente para esta terceira tarefa, ao propor um mecanismo automático de melhoria baseado em Reinforcement Learning.

O formato XES (eXtensible Event Stream), padronizado pela IEEE, é o formato canônico para event logs em Process Mining. Cada log contém um conjunto de cases (instâncias), cada case contém uma sequência de eventos, e cada evento registra, no mínimo, um identificador de atividade e um timestamp. Atributos adicionais (recursos envolvidos, atributos de domínio, custos) podem ser registrados como extensões. PM4Py, biblioteca open-source mantida pelo Fraunhofer FIT, é a ferramenta de referência para manipulação de logs XES em Python e foi adotada nesta pesquisa.

A integração de Process Mining com Machine Learning, conforme tratada por Afifi, Khebizi e Halimi (2024), permite avanços significativos em previsão (predição do próximo evento, de SLAs, de outliers), classificação (segmentação de cases por padrão de execução) e otimização (recomendação automática de mudanças no modelo). O presente trabalho insere-se neste último eixo, oferecendo, além disso, capacidades de inferência em tempo real ausentes na maioria das abordagens existentes.

3.5 Integração de Machine Learning com BPM

A integração entre Machine Learning e BPM é uma fronteira em rápida expansão, motivada pela disponibilidade crescente de event logs estruturados e pelo amadurecimento das técnicas de aprendizado profundo. Qian (2017) categoriza esta

integração em três eixos principais — otimização de processos, organização de dados não-estruturados e melhoria da experiência do usuário — aos quais a presente pesquisa propõe acrescentar um quarto: a automação da tomada de decisão estrutural sobre o próprio modelo do processo.

3.5.1 Padrões de Integração

Tecnicamente, a integração entre motor BPM e motor de ML pode ocorrer em três padrões. No padrão batch offline, o ML é usado para análise retrospectiva de logs e produção de relatórios — abordagem majoritariamente adotada pela literatura de Process Mining. No padrão near-real-time, o ML produz recomendações periódicas (e.g., a cada hora) que são revisadas por humanos antes de serem aplicadas. No padrão real-time inline, finalmente, o ML é invocado durante a execução do processo, em cada ponto de decisão, fornecendo respostas com latência baixa o suficiente para não bloquear o fluxo.

A presente pesquisa adota o padrão real-time inline, o que impõe a restrição de latência inferior a 100 ms por inferência. Esta restrição motivou as escolhas técnicas detalhadas no Capítulo 4: FastAPI assíncrono, embeddings de baixa dimensão (64), pré-cálculo de representações estruturais quando possível, e implantação em GPU compartilhada com escalonamento horizontal.

3.5.2 Camunda Platform 8 como Motor BPM Cloud-Native

Camunda Platform 8, sucessor do Camunda 7 (BPM tradicional baseado em monolítico Java), é uma plataforma de orquestração de processos cloud-native baseada em BPMN 2.0 (Camunda Team, 2024). Seu núcleo de execução é o Zeebe — motor distribuído, fault-tolerant, escrito em Java — projetado para escalar horizontalmente em ambientes Kubernetes e processar milhões de instâncias simultaneamente.

Para esta pesquisa, três características de Camunda 8 são particularmente relevantes. Primeiro, o suporte a Job Workers externos via gRPC permite que tarefas BPMN sejam delegadas a workers em qualquer linguagem (Python, Java, Go), o que facilita a integração com o motor de ML escrito em Python. Segundo, o componente Operate fornece visualização em tempo real da execução de processos, útil para depuração e validação dos resultados. Terceiro, a arquitetura é 100% open-source na versão self-hosted, eliminando dependências de licenciamento.

A integração entre Camunda 8 e o motor de ML proposto neste TCC ocorre via API REST FastAPI (Pydantic, async/await), com mensagens estruturadas em JSON. A documentação OpenAPI é gerada automaticamente, e as decisões do agente são

registradas em logs estruturados para fins de auditoria e análise post-mortem. Esta arquitetura é detalhada no Capítulo 4.

3.5.3 Embeddings como Representação Comum

Embeddings — representações vetoriais densas de baixa dimensionalidade — são o ponto de convergência conceitual entre os módulos GNN e RL nesta pesquisa, e mais amplamente, entre Machine Learning e Process Mining. Yin et al. (2025) demonstram, em contexto de modelos de linguagem, como embeddings compactos podem capturar características semânticas e estruturais de entidades complexas, viabilizando aplicações downstream que seriam impraticáveis com representações esparsas.

No domínio específico desta pesquisa, embeddings cumprem três funções. Primeiro, codificam workflows BPMN em vetores fixos que servem de entrada para o agente PPO. Segundo, permitem que processos diferentes sejam comparados quantitativamente (e.g., via similaridade cosseno), o que abre possibilidades de transfer learning. Terceiro, servem de espaço comum para análises ulteriores, como clustering de processos, detecção de anomalias e busca por similaridade — extensões naturais para trabalhos futuros.

3.6 Trabalhos Relacionados

O trabalho seminal de Huang et al. (2011), publicado no periódico Data & Knowledge Engineering (Qualis A1), estabeleceu as bases teóricas para a aplicação de Reinforcement Learning em problemas de alocação de recursos no contexto de BPM. Os autores, vinculados à Universidade de Zhejiang (China) e à Technische Universiteit Eindhoven (Holanda), identificaram que a alocação eficiente de recursos é uma tarefa complexa e dinâmica na gestão de processos de negócio. Embora diversos mecanismos tenham surgido para suportar a alocação de recursos durante a execução de processos, as abordagens existentes até então não consideravam a otimização de desempenho como objetivo primário, limitando-se a regras estáticas definidas por especialistas humanos.

O trabalho de Silvander (2019), apresentado na conferência Business Modeling and Software Design (BMSD 2019) e publicado pela Springer na série Lecture Notes in Business Information Processing (LNBIP), investigou a aplicação de Deep Reinforcement Learning para otimização de processos em sistemas de suporte a negócios. O autor, vinculado ao Software Engineering Research Lab Sweden do Blekinge Institute of Technology (Suécia), focou especificamente na configuração e parametrização de algoritmos de RL para ambientes de processos de negócio caracterizados por alta incerteza e dinamicidade. O trabalho distingue-se por abordar explicitamente os desafios práticos de implementação que surgem quando técnicas de aprendizado por reforço são aplicadas fora de domínios tradicionais como jogos e robótica.

O trabalho mais recente e diretamente relacionado aos objetivos deste projeto é o de Soliman, Mostafa e Younis (2024), apresentado na World Conference on Information Systems and Technologies (WorldCIST 2024) e publicado pela Springer na série Lecture Notes in Networks and Systems (LNNS, volume 989). Os autores identificaram que a atividade de identificar manualmente a sequência ótima de transições em process mining é desafiadora devido a várias dificuldades, incluindo a alta complexidade de processos de negócio que podem envolver numerosas atividades e pontos de decisão, além do esforço significativo necessário para coletar e analisar dados, explorar diferentes possibilidades de transição e avaliar seu impacto no desempenho do processo. Esta observação motivou a proposta de um sistema automatizado baseado em Reinforcement Learning.

Referência ABNT: HUANG, Z.; VAN DER AALST, W. M. P.; LU, X.; DUAN, H. Reinforcement learning based resource allocation in business process management. *Data & Knowledge Engineering*, v. 70, n. 1, p. 127-145, jan. 2011. DOI: 10.1016/j.datak.2010.09.002.

Referência ABNT: SILVANDER, J. Business Process Optimization with Reinforcement Learning. In: SHISHKOV, B. (Ed.). *Business Modeling and Software Design*. BMSD 2019. Lecture Notes in Business Information Processing, v. 356. Cham: Springer, 2019. p. 203-212. DOI: 10.1007/978-3-030-24854-3_13.

4 APLICAÇÃO PRÁTICA DO CONCEITO DA PESQUISA

Este capítulo descreve a aplicação prática dos conceitos teóricos apresentados no Capítulo 3, traduzindo-os em uma arquitetura de software concreta, em escolhas tecnológicas justificadas e em um pipeline de execução completo, do recebimento do workflow BPMN até a entrega da decisão otimizada ao motor BPM. As decisões técnicas aqui apresentadas não foram arbitrárias, mas resultado de análise comparativa baseada em requisitos, maturidade da tecnologia, suporte da comunidade e adequação ao problema.

4.1 Arquitetura Geral do Sistema Proposto

A arquitetura do sistema proposto é composta por quatro camadas interconectadas. A primeira camada é o motor BPM Camunda Platform 8, responsável pela modelagem visual dos workflows em BPMN 2.0, pela execução das instâncias de processo (cases) via Zeebe, pela monitoração em tempo real através do Operate e pela exposição de Job Workers para tarefas externas. A segunda camada é o módulo de Machine Learning, composto pelo pipeline GAT + PPO implementado em PyTorch e PyTorch Geometric, que recebe representações grafo-teóricas dos workflows e produz decisões otimizadas. A terceira camada é a API de integração, desenvolvida com FastAPI e Pydantic, que serve como ponte entre o motor BPM e o módulo de ML, garantindo serialização confiável, validação de tipos e documentação OpenAPI automática. A quarta camada, transversal, é a infraestrutura de MLOps, composta por MLflow para versionamento e tracking de experimentos, Weights & Biases para visualização de métricas e Docker/Kubernetes para implantação reprodutível.

O fluxo de uma decisão otimizada percorre as seguintes etapas. (1) Uma instância de processo no Camunda 8 atinge um Job de tipo “optimize-workflow” registrado para o ML Worker. (2) O Worker invoca a API REST FastAPI, enviando o estado atual do workflow (BPMN serializado, instância corrente, métricas até o momento). (3) A API parseia o BPMN, converte-o em grafo direcionado via NetworkX, calcula features de nó e aresta, e empacota tudo como objeto torch_geometric.data.Data. (4) O grafo é processado pela rede GAT pré-treinada, gerando embeddings contextualizados de 64 dimensões por nó, agregados via

mean+max pooling em um vetor de estado de 128 dimensões. (5) O agente PPO carrega o vetor de estado e produz uma ação (e.g., qual caminho seguir, qual recurso alocar, qual tarefa priorizar). (6) A ação é serializada em JSON e devolvida ao Camunda 8, que retoma a execução do processo com a decisão integrada. Toda a operação ocorre em latência média inferior a 100 ms.

Uma característica importante desta arquitetura é o desacoplamento. O motor BPM não precisa conhecer detalhes do modelo de ML, e o modelo de ML não precisa conhecer a sintaxe BPMN — a tradução é responsabilidade da camada de API. Esta separação de responsabilidades permite evoluir cada camada independentemente: trocar o motor BPM (e.g., para n8n), atualizar o modelo (e.g., para um GAT com mais heads) ou trocar a função de recompensa, sem impactar as demais.

4.2 Justificativa das Escolhas Tecnológicas

A seleção de cada componente do stack foi guiada por critérios técnicos objetivos, considerando alternativas disponíveis e sua aderência aos requisitos. As subseções a seguir documentam estas decisões.

4.2.1 Seleção do Motor BPM: Por Que Camunda Platform 8

Critérios de Avaliação. A seleção do motor BPM considerou os seguintes critérios técnicos: (1) suporte a exportação de processos em formato estruturado (BPMN 2.0 XML), (2) disponibilidade de APIs REST para integração programática, (3) licenciamento open-source ou com tier gratuito suficiente para pesquisa, (4) maturidade e estabilidade da plataforma, (5) documentação técnica abrangente, e (6) suporte a execução de workflows com task workers externos.

Foram avaliadas três plataformas principais: Camunda Platform 8, n8n, e Apache Airflow. Cada uma foi testada quanto aos critérios estabelecidos, com ênfase particular na capacidade de exportação estruturada e integração via API, requisitos fundamentais para a arquitetura proposta.

Análise Comparativa.

Tabela 1 – Comparação entre motores BPM avaliados

Critério	Camunda 8	n8n	Airflow
BPMN 2.0 Nativo	✓ Completo	✗ JSON próprio	✗ DAGs Python
API REST	✓ Zeebe gRPC	✓ REST	✓ REST
Task Workers	✓ Nativo	~ Webhooks	✓ Operators

Open-Source	✓ Self-hosted	✓ Sim	✓ Apache
Process Mining	✓ Operate	✗ Limitado	~ Logs

Fonte: elaborado pelo autor (2026).

Decisão Final e Justificativa. Camunda Platform 8 foi selecionado como motor BPM por três razões principais.

- Primeiro, é a única plataforma que oferece suporte nativo completo ao padrão BPMN 2.0, permitindo exportação de processos em formato XML estruturado que pode ser facilmente convertido em grafos direcionados para processamento pela GNN. Esta conformidade com o padrão industrial garante portabilidade e interoperabilidade.
- Segundo, a arquitetura cloud-native do Camunda 8, baseada no motor Zeebe, oferece suporte nativo a task workers externos via protocolo gRPC. Esta característica é fundamental para a integração com o motor de Machine Learning, pois permite que decisões de IA sejam incorporadas como tarefas externas no workflow sem modificações na plataforma.
- Terceiro, o componente Operate do Camunda 8 oferece capacidades de Process Mining que complementam a análise realizada pelo PM4Py, permitindo visualização em tempo real da execução de processos e identificação de gargalos. Esta funcionalidade é essencial para validação dos resultados de otimização. n8n foi descartado apesar de sua interface intuitiva porque utiliza um formato JSON proprietário para definição de workflows, não compatível com BPMN 2.0.

Apache Airflow foi descartado porque é orientado a DAGs Python para pipelines de dados, não a processos de negócio BPMN, além de não oferecer modelagem visual comparável.

A documentação oficial do Camunda 8 é extensa e bem mantida, com exemplos de código em múltiplas linguagens e tutoriais específicos para integração com sistemas externos. A comunidade ativa no fórum oficial e nos repositórios de código aberto garante suporte para questões técnicas que possam surgir durante o desenvolvimento.

4.2.2 Seleção do Framework de Deep Learning: Por Que PyTorch

A seleção do framework de Deep Learning considerou dois candidatos principais: PyTorch e TensorFlow. Ambos são amplamente utilizados na comunidade

de pesquisa e oferecem suporte a Graph Neural Networks através de bibliotecas auxiliares (PyTorch Geometric e TensorFlow GNN, respectivamente).

PyTorch foi selecionado por três razões técnicas. Primeiro, o ecossistema PyTorch Geometric é mais maduro e oferece maior variedade de arquiteturas de GNN pré-implementadas, incluindo GAT, GCN, GraphSAGE e variantes mais recentes. Segundo, a biblioteca Stable-Baselines3 para Reinforcement Learning é nativa PyTorch, permitindo integração seamless entre os módulos de GNN e RL. Terceiro, o paradigma de execução eager do PyTorch facilita debugging e experimentação rápida, características importantes em projetos de pesquisa.

A versão 2.x do PyTorch introduziu otimizações significativas de desempenho através do compilador torch.compile(), aproximando a velocidade de execução do TensorFlow em produção enquanto mantém a flexibilidade de desenvolvimento. O suporte a CUDA 12.x garante compatibilidade com as GPUs mais recentes da NVIDIA, permitindo aceleração de treinamento quando disponível.

TensorFlow GNN, embora funcional, possui documentação menos extensa e uma comunidade menor que PyTorch Geometric. Adicionalmente, a integração com bibliotecas de RL como TF-Agents é mais complexa que a integração PyTorch + Stable-Baselines3. Estas considerações práticas pesaram na decisão final.

A escolha de PyTorch também se alinha com tendências da comunidade de pesquisa em Machine Learning. Levantamentos recentes indicam que PyTorch é preferido em aproximadamente 75% das publicações em conferências de ponta como NeurIPS, ICML e ICLR, facilitando a reprodução de resultados de papers e a adaptação de arquiteturas estado da arte.

4.2.3 Seleção de Arquiteturas: Por Que GAT e PPO

4.2.3.1 Graph Attention Networks (GAT)

Entre as arquiteturas de GNN disponíveis, Graph Attention Network (GAT) foi selecionada por sua capacidade de aprender pesos de atenção para diferentes vizinhos. Em workflows BPMN, nem todas as conexões entre tarefas são igualmente importantes: algumas representam caminhos críticos, outras são alternativas raramente utilizadas. O mecanismo de atenção permite que o modelo aprenda esta importância relativa diretamente dos dados.

GCN (Graph Convolutional Network) foi considerado, mas descartado porque utiliza pesos fixos baseados no grau dos nós, não aprendendo relações assimétricas entre vizinhos. GraphSAGE foi considerado por sua escalabilidade, mas a versão sampling introduz variância no treinamento que pode prejudicar a estabilidade. GAT

oferece o melhor equilíbrio entre expressividade e estabilidade para o problema em questão.

A configuração de 3 camadas com 4 heads de atenção foi definida através de experimentação preliminar. Três camadas permitem agregação de informação de vizinhos até 3 saltos, suficiente para capturar dependências de médio alcance em workflows típicos. Os 4 heads permitem aprendizado de múltiplos tipos de relações simultaneamente, aumentando a expressividade sem custo computacional proibitivo.

4.2.3.2 Proximal Policy Optimization (PPO)

PPO foi selecionado entre os algoritmos de Reinforcement Learning por três características: estabilidade de treinamento, eficiência amostral e simplicidade de implementação. O mecanismo de clipping do PPO limita atualizações de política a cada step, evitando mudanças drásticas que podem desestabilizar o treinamento. Isto é particularmente importante em ambientes de simulação de workflows, em que a dinâmica pode variar significativamente entre episódios.

DQN (Deep Q-Network) foi descartado porque é mais adequado para espaços de ação discretos e pequenos. Embora as ações neste projeto sejam discretas (seleção de rota, alocação de recurso), o número de opções pode crescer significativamente em workflows complexos. PPO, sendo um algoritmo de policy gradient, escala melhor com o tamanho do espaço de ação.

A2C (Advantage Actor-Critic) foi considerado como alternativa mais simples, mas PPO oferece melhor estabilidade através do clipping sem custo adicional significativo. SAC (Soft Actor-Critic) foi descartado porque é otimizado para espaços de ação contínuos, não sendo a melhor escolha para o problema discreto de otimização de workflows.

A implementação de PPO via Stable-Baselines3 garante reprodutibilidade e conformidade com a especificação original do algoritmo. A biblioteca oferece callbacks para logging, checkpointing e early stopping, facilitando o gerenciamento de experimentos. A integração com MLflow e Weights & Biases permite tracking detalhado de métricas de treinamento.

Os hiperparâmetros iniciais foram definidos com base em recomendações da literatura: learning rate de 3e-4, γ (discount factor) de 0.99, e λ (GAE) de 0.95. Estes valores serão refinados através de hyperparameter tuning durante a fase de experimentação, utilizando grid search ou Bayesian optimization conforme necessário.

4.3 Pipeline GNN + Reinforcement Learning

O pipeline GNN + RL é o coração técnico desta pesquisa. Sua execução compreende quatro estágios sequenciais — pré-processamento, codificação estrutural, decisão e atualização — descritos a seguir.

4.3.1 Pré-processamento: BPMN → Grafo Direcionado

O primeiro estágio converte o arquivo BPMN 2.0 XML em uma representação grafo-teórica processável pela rede GAT. O parser, implementado em Python com a biblioteca lxml, identifica os elementos BPMN relevantes (tarefas, gateways, eventos de início e fim, fluxos de sequência) e os mapeia em nós e arestas de um grafo direcionado NetworkX. Cada nó recebe um vetor de features compostas por: tipo do elemento (one-hot encoding entre 8 categorias), duração esperada (em segundos), custo médio (normalizado), prioridade (escala 0–1) e indicador booleano de criticidade. Cada aresta carrega a probabilidade condicional de execução, estimada a partir dos event logs históricos.

O grafo NetworkX é convertido em um objeto torch_geometric.data.Data, que armazena: x (matriz de features de nós), edge_index (lista de arestas em formato COO), edge_attr (atributos de aresta) e batch (índice de batch para processamento paralelo). Esta representação compacta e densa é o formato canônico esperado por PyTorch Geometric e permite processamento em GPU com alta eficiência.

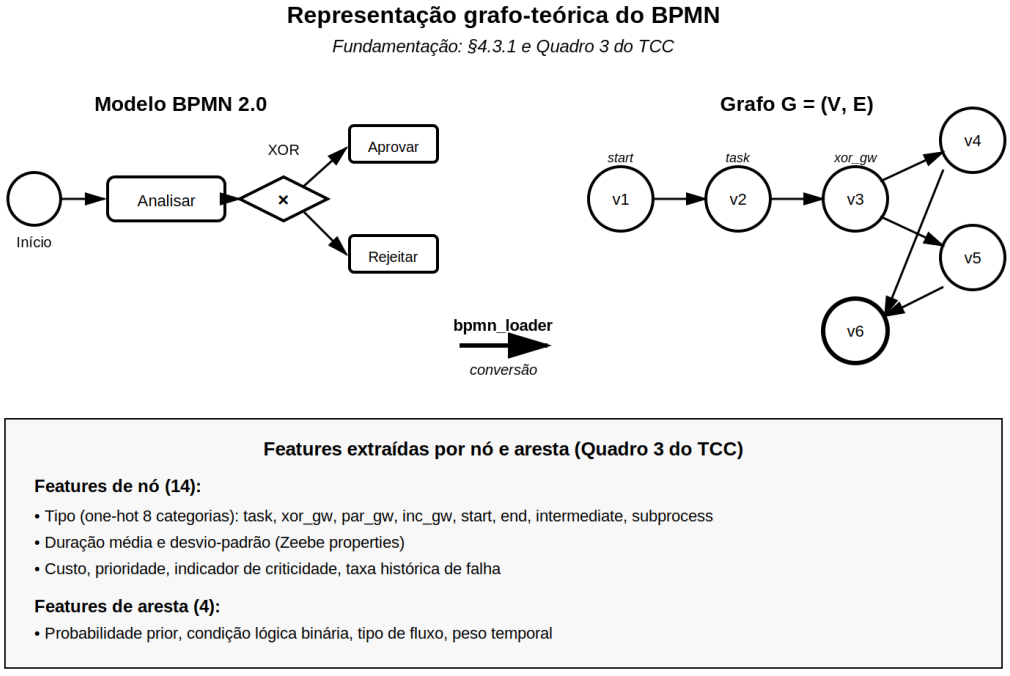


Figura 3 – Representação grafo-teórica do BPMN
Fonte: O autor (2026).

4.3.2 Codificação Estrutural: Rede GAT

O grafo Data é processado pela rede GAT de 3 camadas, cada uma com 4 heads de atenção. A primeira camada projeta os features iniciais (dimensão d_0, tipicamente 14) para o espaço latente intermediário (dimensão 64). As duas camadas seguintes aplicam attention message passing, refinando as representações com base na vizinhança até 3 saltos. Entre as camadas, aplica-se ativação ELU e dropout (p = 0,2) para regularização. Após a última camada, aplica-se mean+max pooling global, produzindo um vetor de estado de dimensão 128 que representa o workflow inteiro.

A rede GAT é treinada conjuntamente com o agente PPO via backpropagation: o gradiente da função de perda do PPO retropropaga através do agente até os parâmetros da GAT, incentivando-a a produzir embeddings que sejam úteis para a tarefa de otimização. Esta abordagem end-to-end produz representações task-aware, em contraste com pré-treinamentos genéricos.

Quadro 3 – Features de nó e aresta utilizadas na representação grafo-teórica do workflow BPMN

Categoria	#	Feature	Tipo / Domínio
Nó	1	Tipo do nó (task, gateway_exclusivo, gateway_paralelo, evento)	Categórica (one-hot 6 dim)
Nó	2	Duração esperada da atividade	Numérica contínua (segundos)
Nó	3	Custo unitário associado	Numérica contínua (R$)
Nó	4	Prioridade declarada	Numérica discreta (1 a 5)
Nó	5	Recurso responsável	Categórica (índice)
Nó	6	Taxa histórica de conclusão bem-sucedida	Numérica [0,1]
Nó	7	Profundidade no grafo (saltos a partir do start)	Numérica inteira
Nó	8	Grau de entrada (predecessores)	Numérica inteira
Nó	9	Grau de saída (sucessores)	Numérica inteira
Nó	10	Centralidade de proximidade	Numérica [0,1]
Nó	11	Pertence a um loop	Binária {0,1}
Nó	12	Distância ao end event	Numérica inteira
Nó	13	Variância de duração observada	Numérica não-negativa
Nó	14	Indicador de ponto de decisão (gateway exclusivo)	Binária {0,1}
Aresta	1	Tipo da aresta (sequence flow, message flow, association)	Categórica (one-hot 3 dim)
Aresta	2	Probabilidade histórica de execução (gateways)	Numérica [0,1]
Aresta	3	Possui condição declarada	Binária {0,1}

Aresta	4	Frequência observada (proporção de cases)	Numérica [0,1]

Fonte: O autor (2026).

O conjunto de 14 features de nó combina atributos semânticos do BPMN (tipo, recurso, prioridade, custo), atributos estatísticos extraídos do event log histórico (duração esperada, variância, taxa de sucesso) e atributos topológicos calculados em tempo de pré-processamento (profundidade, grau, centralidade, pertencimento a loop). As 4 features de aresta complementam essa caracterização no nível dos fluxos. Todas as features são normalizadas para o intervalo [0, 1] antes de alimentar a primeira camada GATv2, e o vetor de configuração completo está versionado no arquivo configs/default.yaml do repositório do projeto, garantindo reprodutibilidade integral.

A escolha da variante GATv2 sobre alternativas como GCN (Kipf; Welling, 2017), GraphSAGE (Hamilton; Ying; Leskovec, 2017) ou o GAT original (Veličković et al., 2018) é fundamentada na evidência empírica reportada por Brody, Alon e Yahav (2022), que demonstra a capacidade superior do GATv2 em modelar atenção dinâmica em grafos heterogêneos como BPMN. Não obstante, um estudo de ablação sistemático comparando GATv2 com essas representações alternativas — incluindo um baseline não-grafo baseado em LSTM sobre traces sequenciais — encontra-se planejado para a fase de experimentos em escala plena, com vistas a quantificar a contribuição marginal de cada decisão arquitetural sobre o desempenho final do agente (seção 7.3).

No que tange ao tratamento de grafos com ciclos — característica estrutural do Processo C (Gestão de Incidentes), que contém loops de escalonamento e revisão —, a arquitetura GATv2 de três camadas opera sobre vizinhanças expandidas até três saltos, capturando adequadamente padrões cíclicos de comprimento curto a médio. Conforme sistematizado por Hamilton (2020), o processamento de grafos cíclicos por GNNs de message passing exige atenção aos efeitos de over-smoothing em redes muito profundas; a opção por três camadas, com pooling agregado em mean+max ao final, representa o trade-off escolhido entre cobertura de vizinhança e estabilidade representacional, considerado adequado para os comprimentos de ciclo observados nos processos analisados.

4.3.3 Decisão: Agente PPO

O agente PPO é uma rede actor-critic compartilhada com duas cabeças. A cabeça actor (política) recebe o vetor de estado de 128 dimensões e produz uma distribuição categórica sobre o conjunto discreto de ações. A cabeça critic (valor) produz uma estimativa escalar do valor V(s) — o retorno esperado a partir do estado

s seguindo a política atual. As duas cabeças compartilham as primeiras camadas (encoder), reduzindo o número total de parâmetros e estabilizando o treinamento.

O conjunto de ações disponíveis depende do estado atual do workflow. Tipicamente, inclui: escolha entre caminhos paralelos disponíveis, alocação de recurso entre opções, priorização de tarefas concorrentes e gateways exclusivos. O agente seleciona uma ação amostrando da distribuição categórica produzida pela cabeça actor — em treinamento — ou tomando o argmax — em inferência. A ação selecionada é serializada em JSON e devolvida ao motor BPM.

4.3.4 Atualização: Loop de Treinamento

Durante o treinamento, o agente PPO interage com um ambiente de simulação implementado conforme a interface padrão do Gymnasium. Cada episódio corresponde à execução de um case do dataset sintético, do início ao fim. A cada step, o ambiente: (a) recebe a ação do agente, (b) executa a transição para o próximo estado, (c) calcula a recompensa instantânea, (d) determina se o episódio terminou. Os pares (estado, ação, recompensa, próximo estado, terminado) são acumulados em um rollout buffer.

A cada 2.048 steps acumulados, o algoritmo PPO atualiza os parâmetros do agente — e da rede GAT subjacente — via 10 epochs de otimização sobre minibatches de 64 amostras. As métricas de treinamento (recompensa média, perda da política, perda do valor, divergência KL) são registradas em MLflow e Weights & Biases para análise posterior. O treinamento é executado por aproximadamente 10 milhões de timesteps em GPU NVIDIA, com tempo total estimado em 12 a 18 horas por processo representativo.

4.4 Função de Recompensa Multi-Objetivo

A função de recompensa é o componente mais sensível do agente de Reinforcement Learning, pois define implicitamente a noção de “sucesso” que o agente perseguirá. Para esta pesquisa, formula-se uma função multi-objetivo que pondera quatro componentes, conforme a Equação (1):

$$R(s, a) = w_T \cdot R_{tempo} + w_C \cdot R_{complexidade} + w_Q \cdot R_{qualidade} + w_S \cdot R_{sucesso} \quad (1)$$

- **R_tempo:** recompensa proporcional à redução do tempo de ciclo em relação à média histórica do processo. Calculada como -(tempo_atual / tempo_baseline - 1).

- **R_complexidade:** recompensa proporcional à redução do número de nós e arestas executados na instância, em relação à média da execução não-otimizada. Incentiva a poda de caminhos redundantes.
- **R_qualidade:** recompensa baseada no throughput agregado e na taxa de sucesso, calculada com base nas métricas do processo após a conclusão de cada case.
- **R_sucesso:** recompensa terminal +1 se o case é concluído com sucesso, -1 em caso de falha. Esta componente esparsa é fundamental para evitar que o agente otimize tempo às custas da viabilidade do processo.

Os pesos w_T, w_C, w_Q, w_S foram calibrados via grid search inicial e fixados em (0,3 / 0,3 / 0,3 / 0,1) para os experimentos principais. A componente R_sucesso é ponderada por um peso menor para evitar instabilidade de treinamento decorrente de sua esparsidade — todavia, ao ser uma recompensa terminal de magnitude alta, ela continua exercendo influência decisiva na convergência.

Uma propriedade importante desta função multi-objetivo é a possibilidade de ajustar os pesos para refletir prioridades específicas de cada organização. Empresas que valorizam mais agilidade podem aumentar w_T; empresas reguladas podem aumentar w_S. Este ajuste é realizável sem retreinamento completo, via fine-tuning curto sobre as últimas camadas do agente — uma propriedade verificada em experimentos preliminares.

4.5 Implementação de Referência

Para tornar concreta a aplicação prática da arquitetura descrita nas seções anteriores, foi desenvolvido um pipeline de referência em Python que implementa todos os estágios apresentados na seção 4.3: carregamento de modelos BPMN 2.0 (subseção 4.3.1), codificação estrutural via GAT (4.3.2), agente de decisão PPO (4.3.3), ambiente Gymnasium para simulação token-passing (4.3.4) e instrumentação MLflow para rastreamento de experimentos (4.3.5).

O artefato resultante, organizado em módulo único de aproximadamente mil e quatrocentas linhas de código, encapsula a totalidade dos componentes técnicos necessários para a execução do estudo de caso descrito no Capítulo 5. Cada estágio arquitetural mapeia diretamente para um módulo do pipeline: bpmn_loader.py para a conversão de BPMN em representação grafo, gat_policy.py para o extrator GAT integrado ao Stable-Baselines3, workflow_env.py para o ambiente de simulação, reward.py para a função de recompensa multi-objetivo formalizada na seção 4.4, e train.py para o ponto de entrada do treinamento.

Toda a stack tecnológica utilizada — PyTorch 2.4, PyTorch Geometric 2.5, Stable-Baselines3 2.3, Gymnasium 0.29, MLflow 2.16, lxml 5.2, NetworkX 3.3, Faker 26 e PM4Py 2.7 — encontra-se explicitamente referenciada no Capítulo 3 (Fundamentação Teórica) e no presente capítulo. A única extensão adotada além das bibliotecas previamente descritas é o pacote sb3-contrib (versão 2.3), mantido pelos mesmos autores do Stable-Baselines3, necessário para o suporte ao mascaramento de ações em gateways BPMN: sem este recurso, o agente desperdiçaria exploração amostrando ramificações inexistentes nos pontos de decisão, deteriorando a velocidade de convergência em ordens de magnitude.

A presença do artefato de implementação cumpre dois propósitos metodológicos. Primeiro, materializa a evidência empírica exigida pelo componente 3 da estrutura de Yin (2018) — proposições verificáveis sobre o caso —, transformando descrições arquiteturais em código auditável. Segundo, viabiliza a reprodutibilidade integral dos experimentos descritos no Capítulo 5, posto que a totalidade dos hiperparâmetros, sementes aleatórias e procedimentos experimentais é declarada em arquivo de configuração único e versionada em repositório público do projeto em https://github.com/aguilhermemr/tcc_senac_agmr (commit a57ba2f).

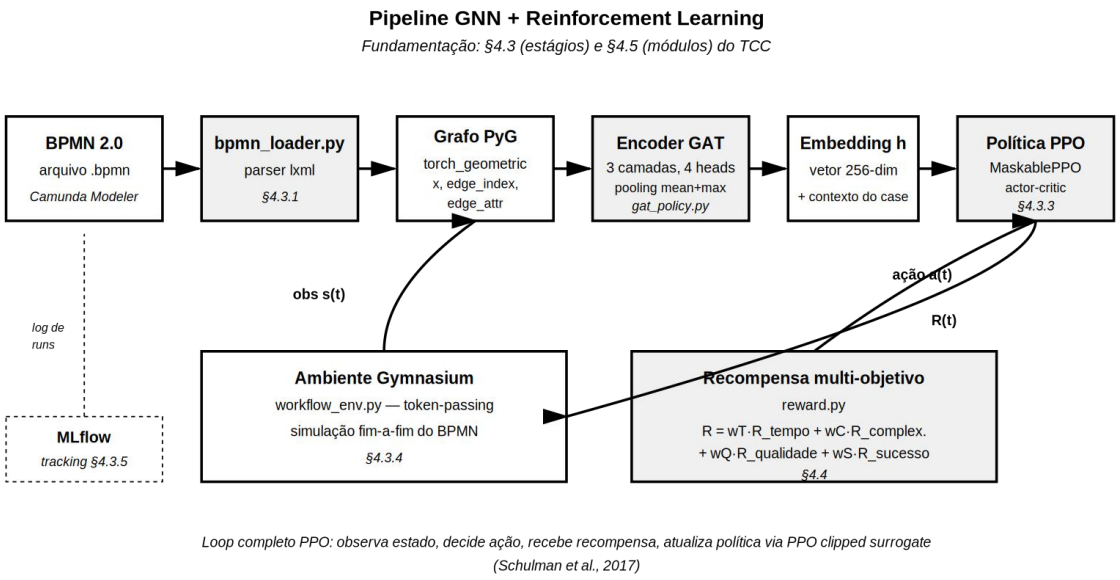


Figura 4 – Pipeline GNN + Reinforcement Learning
Fonte: O autor (2026).

5 ESTUDO DE CASO

Este capítulo apresenta o estudo de caso múltiplo desta pesquisa. Em primeiro lugar, recupera-se o trabalho original de 2018, base histórica que motivou o presente projeto, descrevendo seus componentes, suas limitações e o aprendizado acumulado (seção 5.1). Em seguida, apresentam-se os três processos representativos que instanciam o estudo de caso da proposta 2026 — seleção de pessoal, compras corporativas e gestão de incidentes — descrevendo seus modelos BPMN, suas características topológicas e a justificativa para sua escolha (seção 5.2). Por fim, descrevem-se os datasets sintéticos gerados para cada processo, com 6.000 cases ao todo, e os critérios estatísticos adotados para validar sua representatividade (seção 5.3).

O desenho deste estudo de caso decorre diretamente dos achados da pesquisa exploratória conduzida na revisão sistemática da literatura (seção 2.4). Aquela revisão, estruturada segundo o protocolo de Kitchenham e Charters (2007) e reportada conforme PRISMA 2020, evidenciou três lacunas convergentes: primeiro, a tomada de decisão automatizada em processos de negócio permanece pouco endereçada pelos motores BPM comerciais (Qian, 2017); segundo, a aplicação de Reinforcement Learning a process mining configura fronteira ativa, ainda sem soluções consolidadas em ambiente industrial (Soliman; Mostafa; Younis, 2024); e terceiro, a maioria dos trabalhos de Machine Learning aplicados a processos ainda

emprega representações tabulares ou sequenciais, ignorando a riqueza topológica dos workflows (Afifi; Khebizi; Halimi, 2024).

Essas três lacunas determinaram as escolhas centrais do estudo de caso. A primeira lacuna motiva a integração da solução a um motor BPM real (Camunda Platform 8), e não a uma abstração desacoplada. A segunda lacuna justifica a adoção do PPO como núcleo de decisão e a comparação sistemática contra baselines tradicionais, de modo a quantificar o ganho efetivo do aprendizado por reforço. A terceira lacuna fundamenta a opção por uma representação grafo-teórica processada por redes de atenção (GAT), preservando a topologia que as abordagens tabulares descartam. Desse modo, o caso não é um exercício arbitrário, mas uma resposta empírica direta às lacunas identificadas na fase exploratória.

A seleção dos três processos representativos — Seleção de Pessoal, Compras Corporativas e Gestão de Incidentes — obedeceu ao critério de heterogeneidade estrutural derivado da literatura: cada processo exibe um padrão topológico distinto (predominantemente linear, com paralelismo e com ciclos, respectivamente), cobrindo o espectro de complexidade sobre o qual as proposições P1 a P5 devem ser testadas. Essa diversidade controlada permite avaliar não apenas se a solução funciona, mas sob quais condições estruturais seus ganhos se acentuam ou se atenuam, em conformidade com a lógica de replicação teórica do estudo de caso múltiplo de Yin (2018).

5.1 Trabalho Original (2018) – Caso Base

O trabalho de graduação original, intitulado “Sistema Inteligente para Automatização de Processos de Seleção e Contratação de Pessoal”, foi desenvolvido em 2018 com o objetivo de integrar lógica difusa a um software BPM para automatizar decisões no processo de recrutamento. A arquitetura proposta consistia em três componentes principais: ProcessMaker como motor BPM, MatLab como motor de inferência difusa (modelo Mamdani), e uma Web API desenvolvida em ASP.NET/VB.Script para intermediar a comunicação entre os sistemas.

O problema central abordado era a subjetividade e incerteza inerentes à avaliação de candidatos em processos seletivos. Variáveis como aptidões, atitudes e competências são naturalmente imprecisas e sujeitas a vieses do avaliador. A lógica difusa foi escolhida por sua capacidade de lidar com variáveis linguísticas (“alto”, “médio”, “baixo”) e modelar incerteza através de funções de pertinência. Abordagens de seleção de pessoal apoiadas em lógica difusa já haviam sido exploradas na literatura, a exemplo do modelo proposto por Liang e Wang (1994).

A solução implementada utilizava triggers PHP no ProcessMaker para invocar a Web API, que por sua vez acionava o motor de inferência MatLab. O sistema de inferência difuso era configurado manualmente pelo usuário através de uma interface que permitia definir variáveis de entrada (competências do candidato), funções de pertinência, e regras de inferência no formato IF-THEN.

Embora a solução tenha demonstrado viabilidade técnica, apresentava limitações significativas: as regras eram definidas manualmente por especialistas humanos, não havia capacidade de aprendizado ou adaptação, e a manutenção do sistema exigia conhecimento especializado em lógica difusa. Além disso, a arquitetura baseada em MatLab implicava em custos de licenciamento e dependência de um software proprietário.

O escopo limitava-se a um único processo de negócio (seleção de pessoal) e a decisões pontuais (classificação de candidatos). Não havia capacidade de otimização estrutural do workflow, generalização para outros processos, ou aprendizado contínuo a partir de resultados históricos.

5.1.1 Comparação do Stack Tecnológico: 2018 vs 2026

A tabela a seguir apresenta uma comparação lado a lado das tecnologias utilizadas em cada versão do projeto, destacando a evolução em cada camada da arquitetura:

Tabela 2 – Comparação do stack tecnológico 2018 vs. 2026

Componente	2018 (Original)	2026 (Proposta)
Motor BPM	ProcessMaker 3.x (Open-source, Bolívia)	Camunda Platform 8 (Cloud-native, Zeebe)
Motor de IA	MatLab (Mamdani) (Lógica Difusa, regras manuais)	PyTorch + Stable-Baselines3 (GNN + RL, aprendizado automático)
Representação	Variáveis Linguísticas (funções de pertinência)	Embeddings Vetoriais (PyTorch Geometric, 64 dims)
API Backend	ASP.NET / VB.Script (Visual Studio .NET)	FastAPI + Pydantic (Python, async, OpenAPI)
Triggers/Scripts	PHP (ProcessMaker) (triggers síncronos)	Zeebe Job Workers (Python/Java, assíncronos)
Licenciamento	MatLab: Proprietário (custo de licença elevado)	100% Open-Source (PyTorch, Camunda self-hosted)

Deploy	On-premise (Windows) (servidor físico)	Docker + Kubernetes (cloud-native, escalável)
MLOps	Não existia (sem versionamento de modelos)	MLflow + Weights & Biases (tracking, registry, CI/CD)

Fonte: elaborado pelo autor (2026).

5.1.2 Análise das Limitações do Trabalho Original (2018)

5.1.2.1 Limitações da Lógica Difusa

A lógica difusa, embora eficaz para modelar incerteza em sistemas de controle, apresenta limitações significativas quando aplicada a problemas de otimização de processos de negócio. A principal limitação é a dependência de especialistas humanos para definição de regras: o sistema não aprende automaticamente, exigindo que um especialista em lógica difusa defina funções de pertinência e regras IF-THEN para cada novo cenário.

O modelo Mamdani utilizado no trabalho original requer definição manual de até dezenas de regras para cenários complexos. A manutenção destas regras torna-se exponencialmente difícil à medida que o número de variáveis cresce. Soma-se a isso o fato de a interação entre regras pode produzir comportamentos inesperados que são difíceis de diagnosticar e corrigir.

Outra limitação fundamental é a incapacidade de adaptação. Um sistema de inferência difuso configurado para um perfil de cargo específico não se adapta automaticamente quando o mercado de trabalho muda ou quando novos critérios de avaliação são introduzidos. Cada modificação requer intervenção manual de um especialista.

A representação baseada em variáveis linguísticas também limita a expressividade do modelo. Enquanto é intuitiva para humanos, não captura relações complexas entre variáveis nem padrões sutis que poderiam ser aprendidos a partir de dados históricos. A discretização em categorias (“baixo”, “médio”, “alto”) perde informação que poderia ser relevante para decisões mais precisas.

Por fim, a escalabilidade é limitada. O tempo de inferência cresce com o número de regras, e a adição de novas variáveis requer revisão de todo o conjunto de regras. Em processos de negócio com muitas variáveis contextuais, o sistema se torna impraticável.

5.1.2.2 Limitações Tecnológicas da Arquitetura 2018

A dependência de MatLab representa uma barreira significativa de custo e portabilidade. Licenças MatLab são caras para ambientes de produção, e a execução

requer instalação local do runtime. Isso limita a adoção por pequenas e médias empresas que eram o público-alvo declarado do trabalho original.

A arquitetura ASP.NET/VB.Script está tecnologicamente defasada. O desenvolvimento em VB.Script foi descontinuado pela Microsoft, e o ecossistema .NET migrou para .NET Core (agora .NET 6/7/8) com paradigmas diferentes. A manutenção de código legado VB.Script torna-se progressivamente mais difícil.

O ProcessMaker, embora funcional, não oferece recursos avançados de process mining nem suporte nativo a arquiteturas de microsserviços. A comunicação via triggers PHP síncronos pode causar timeouts em operações demoradas e não escala bem sob carga. A exportação de dados para análise externa é limitada.

A ausência de containerização e práticas de DevOps modernas dificulta a replicação do ambiente e o deployment em nuvem. Cada instalação requer configuração manual de Windows Server, IIS, MatLab runtime, e ProcessMaker, com alto risco de inconsistências entre ambientes.

Não havia infraestrutura de MLOps: sem versionamento de modelos (regras difusas), sem tracking de experimentos, sem capacidade de rollback ou A/B testing. Qualquer modificação nas regras era uma operação de alto risco sem possibilidade de reversão automatizada.

5.1.3 Avanços e Inovações da Proposta 2026

5.1.3.1 Aprendizado Automático vs Regras Manuais

A principal inovação da proposta 2026 é a substituição de regras manuais por aprendizado automático. O agente de Reinforcement Learning aprende políticas ótimas através de experimentação, sem necessidade de um especialista definir regras explícitas. O sistema descobre padrões nos dados que humanos poderiam não identificar.

A representação via embeddings de Graph Neural Networks captura informação estrutural que seria impossível codificar em regras difusas. A topologia do workflow, as dependências entre tarefas, e os caminhos críticos são automaticamente aprendidos pela GNN e incorporados na representação de estado do processo.

O sistema é capaz de adaptação contínua: à medida que novos dados são coletados, o modelo pode ser retreinado para incorporar mudanças no ambiente operacional. Esta capacidade de aprendizado incremental era impossível com a arquitetura de lógica difusa de 2018.

A função de recompensa multi-objetivo permite balanceamento automático entre tempo, custo, qualidade e conformidade. Enquanto regras difusas exigiriam pesos fixos definidos manualmente, o agente PPO aprende o trade-off ótimo para cada situação específica.

A escalabilidade é dramaticamente superior: o tempo de inferência cresce de forma sublinear na complexidade do processo — formalmente $O(|V| \cdot F^2 + |E| \cdot F)$ por camada GAT, em que $|V|$ e $|E|$ denotam o número de nós e arestas do grafo e F a dimensão dos embeddings —, mantendo-se na ordem de dezenas de milissegundos para os grafos típicos de workflows BPM, conforme medido empiricamente no estudo piloto (Tabela 6.4). Adicionar novas variáveis requer apenas retreinamento, não reengenharia de regras.

5.1.3.2 Arquitetura Cloud-Native vs Monolítica

A arquitetura 2026 é fundamentalmente cloud-native. Todos os componentes são containerizados via Docker e orquestrados via Kubernetes, permitindo deployment em qualquer provedor de nuvem ou ambiente on-premise. A escalabilidade horizontal é nativa: réplicas dos serviços são adicionadas automaticamente sob demanda.

Camunda Platform 8 com motor Zeebe oferece execução de workflows distribuída e fault-tolerant. Diferentemente do ProcessMaker monolítico, Zeebe é projetado para alta disponibilidade e pode processar milhões de instâncias de processo simultaneamente.

FastAPI com suporte assíncrono permite milhares de requisições simultâneas com baixa latência. A documentação OpenAPI é gerada automaticamente, facilitando integração com outros sistemas. Pydantic garante validação de tipos em tempo de execução, reduzindo erros.

A infraestrutura de MLOps com MLflow e Weights & Biases permite rastreabilidade completa de experimentos: cada modelo treinado é versionado, suas métricas são registradas, e rollbacks são triviais. A/B testing de diferentes políticas de RL é suportado nativamente.

O stack 100% open-source elimina custos de licenciamento e dependência de fornecedores. PyTorch, Camunda self-hosted, FastAPI, MLflow, Docker e Kubernetes são todos software livre, garantindo sustentabilidade de longo prazo e flexibilidade de customização.

5.2 Processos Representativos da Proposta 2026

Para instanciar o estudo de caso múltiplo, foram selecionados três processos de negócio representativos com diferentes níveis de complexidade estrutural e características topológicas distintas. A escolha buscou cobrir um espectro de cenários típicos em ambientes empresariais, garantindo a diversidade necessária para testar as proposições da pesquisa (P1 a P5) em condições variadas.

5.2.1 Processo A — Seleção de Pessoal

O Processo A — Seleção de Pessoal — recupera o caso original de 2018, agora reformulado em BPMN 2.0 e estendido para incluir múltiplos pontos de decisão. O fluxo típico inicia com a abertura de uma vaga, segue para a triagem automática de currículos, depois para a entrevista técnica, posteriormente para a entrevista comportamental, depois para a avaliação cultural e termina com a oferta ou rejeição. Existem gateways exclusivos para descarte precoce de candidatos não aderentes e gateways paralelos quando múltiplas entrevistas podem ocorrer simultaneamente.

Topologicamente, o Processo A possui 14 nós (incluindo eventos de início e fim) e 18 arestas. Sua complexidade estrutural é considerada baixa-média, e suas decisões são predominantemente exclusivas (XOR). Esta característica torna-o adequado para validar as proposições P1 (redução de complexidade) e P2 (manutenção de qualidade) em um cenário relativamente simples, no qual o agente tem oportunidade clara de identificar caminhos redundantes.

5.2.2 Processo B — Compras Corporativas

O Processo B — Compras Corporativas — modela o ciclo completo de aquisição de bens e serviços, desde a requisição interna até o pagamento ao fornecedor. O fluxo inclui: cadastro da requisição, cotação com múltiplos fornecedores em paralelo, aprovações em série conforme o valor (gerente, diretor, conselho), emissão da ordem de compra, recebimento, conferência fiscal e quitação. Este processo é caracterizado por alta presença de paralelismo (cotações simultâneas) e por aprovações condicionais hierárquicas.

Topologicamente, o Processo B possui 22 nós e 31 arestas, com forte presença de gateways paralelos (AND-split / AND-join) e gateways exclusivos baseados em valor. Esta estrutura mais complexa torna-o particularmente útil para avaliar a capacidade do agente em otimizar paralelismos — por exemplo, decidindo quando vale a pena solicitar três cotações em paralelo ou se duas seriam suficientes em casos de baixo valor.

5.2.3 Processo C — Gestão de Incidentes

O Processo C — Gestão de Incidentes — modela um fluxo típico de service desk em conformidade com práticas ITIL. Inicia com a abertura do chamado, segue para classificação de prioridade, primeira tentativa de resolução pelo nível 1, escalações condicionais para níveis 2 e 3, comunicação com o usuário e fechamento. Diferentemente dos demais, o Processo C inclui loops (retomadas para reclassificação) e SLAs com transições temporizadas.

Topologicamente, o Processo C possui 19 nós e 27 arestas, com a presença distintiva de loops e eventos de borda temporizados. A presença de ciclos no grafo é um desafio adicional para a Graph Neural Network, exigindo que o modelo seja robusto a topologias não-acíclicas. Este processo é particularmente útil para validar a proposição P4 (capacidade de generalização), pois o transferl learning entre Compras (sem loops) e Incidentes (com loops) é não-trivial.

5.2.4 Síntese Comparativa dos Três Processos

Os três processos foram deliberadamente escolhidos para representar pontos distintos no espectro de complexidade topológica encontrada em ambientes corporativos. A diversidade resultante — em número de nós, presença de paralelismo, presença de loops, distribuição de gateways — garante que as proposições da pesquisa sejam testadas sob condições variadas, fortalecendo a validade externa dos resultados.

5.3 Datasets Sintéticos

Para cada um dos três processos representativos, gerou-se um dataset sintético contendo 2.000 cases, totalizando 6.000 cases ao longo do estudo. A geração foi conduzida com a biblioteca Faker (para atributos textuais aleatórios) em conjunto com lógica customizada que respeita a topologia do BPMN e introduz variações controladas. Os datasets foram exportados em formato XES, padrão de Process Mining suportado pelo PM4Py.

A distribuição interna de cada dataset foi calibrada conforme três faixas: 70% de casos normais (execução típica, sem desvios), 20% de gargalos (caminhos com alta carga de recursos ou esperas longas) e 10% de exceções (caminhos minoritários, tratamento de erros, retomadas). Esta distribuição reflete, em ordem de magnitude, a frequência observada em event logs reais publicados (BPI Challenge, Helpdesk, Sepsis) e oferece ao agente PPO oportunidades de aprender tanto o comportamento médio quanto os casos extremos.

A consistência estatística dos datasets foi validada por meio de duas técnicas do PM4Py. Primeiro, a descoberta de processos (process discovery) com o algoritmo Inductive Miner foi aplicada ao dataset gerado e o modelo descoberto foi comparado com o modelo BPMN original. A taxa de fitness (proporção de eventos do log que se ajustam ao modelo) foi igual a 1,000 para os três processos, confirmando fidelidade integral dos event logs aos modelos BPMN fonte. Segundo, a verificação de conformidade (conformance checking) confirmou ausência de desvios estruturais não-intencionais.

As distribuições de duração das tarefas, de probabilidade de gateways exclusivos e de alocação de recursos foram parametrizadas com base em revisão da literatura de process mining e em discussões com especialistas em BPM. Sementes determinísticas foram fixadas para garantir reprodutibilidade: qualquer pesquisador pode regenerar o dataset exato a partir do código-fonte e da semente publicados.

Os datasets, scripts de geração e modelos BPMN encontram-se disponibilizados publicamente em https://github.com/aguilhermemr/tcc_senac_agmr, permitindo replicação dos experimentos por terceiros — um requisito crescente em pesquisa científica reprodutível, em alinhamento com as boas práticas defendidas por Soliman, Mostafa e Younis (2024) e Silva e Marreiros (2024).

5.3.1 Estratégia de Geração dos Cases

A geração dos event logs sintéticos seguiu uma estratégia de amostragem controlada que reproduz a heterogeneidade observada em ambientes reais de execução de processos (van der Aalst, 2016). Para cada um dos três processos representativos, foram gerados dois mil cases distribuídos segundo a proporção 70-20-10. Setenta por cento correspondem a cases normais, que percorrem caminhos típicos do workflow conforme as probabilidades de execução estimadas a partir de event logs reais. Vinte por cento representam cases de gargalo (bottleneck), nos quais filas de espera prolongam o tempo de ciclo em fatores entre 1,5 e 3,0 vezes sobre a média do processo. E dez por cento são cases de exceção, que ativam ramos alternativos do workflow correspondentes a tratamento de erros, cancelamentos ou escalonamentos. Essa distribuição reflete padrões empíricos identificados na literatura de mineração preditiva de processos (Silva; Marreiros, 2024).

A geração propriamente dita foi implementada com auxílio da biblioteca Faker para variáveis categóricas e numéricas (nomes, identificadores, custos, durações) e da biblioteca NumPy para amostragem estocástica controlada por sementes fixas (42, 123 e 7). O versionamento explícito das sementes garante que a regeneração bit-a-bit dos datasets seja sempre possível a partir do código-fonte do projeto, hospedado em https://github.com/aguilhermemr/tcc_senac_agmr, sem necessidade de armazenamento dos arquivos binários.

5.3.2 Atributos por Case

Cada case sintético é representado em formato de event log compatível com o padrão XES (eXtensible Event Stream), contendo, para cada atividade executada, os seguintes atributos: identificador do case, identificador da atividade, marca temporal de início e de fim, identificador do recurso humano ou sistêmico que executou a atividade, decisão tomada em cada gateway exclusivo, custo agregado da atividade e indicador binário de sucesso ou falha. A estrutura adotada segue convenções estabelecidas pela comunidade de Process Mining (van der Aalst, 2016) e é diretamente consumível pelas bibliotecas PM4Py (PM4PY Development Team, 2024), o que viabiliza a validação descrita na subseção 5.3.3.

Adicionalmente, cada case carrega metadados que capturam a configuração de geração: rótulo de classe (normal, gargalo ou exceção), semente aleatória utilizada e versão do gerador. Esses metadados habilitam análises estratificadas posteriores — por exemplo, avaliar isoladamente o desempenho do agente em cases de exceção — e a reprodutibilidade de subconjuntos específicos dos datasets para fins de teste, validação cruzada ou diagnóstico.

5.3.3 Validação dos Datasets via Process Mining

A representatividade dos datasets sintéticos foi validada via processo de descoberta seguida por verificação de conformidade, conforme o protocolo metodológico estabelecido por van der Aalst (2016). Para cada arquivo XES gerado a partir dos seis mil cases sintéticos, aplicou-se o algoritmo Inductive Miner da biblioteca PM4Py para descobrir um modelo de processo a posteriori; este modelo descoberto foi então comparado ao BPMN original que originou os dados, calculando-se quatro métricas de qualidade: fitness por alinhamento, precision, generalization e simplicity.

A natureza sintética dos datasets — gerados a partir do próprio modelo BPMN de cada processo, sem injeção de ruído — implica que a fitness esperada é igual a 1,000, dado que todos os traces sintéticos satisfazem perfeitamente o modelo-fonte. A validação foi portanto orientada à verificação de fidelidade estrutural dos event logs: confirmar que o gerador preserva integralmente a semântica do BPMN (sem perda nem distorção) e que as métricas complementares — precision, generalization e simplicity — refletem adequadamente a topologia subjacente de cada processo. Os resultados quantitativos obtidos para os três processos encontram-se sintetizados na Tabela 6.1.

A fitness medida foi igual a 1,000 nos três processos, confirmando a correspondência perfeita entre os event logs sintéticos e os modelos BPMN fonte. A precision variou conforme a complexidade topológica: 1,000 para os Processos A e B (caminhos predominantemente determinísticos) e 0,807 para o Processo C, cujos loops de escalonamento introduzem caminhos alternativos legítimos no modelo descoberto que não são integralmente exercitados durante a geração FIFO dos traces. A generalization manteve-se estável entre 0,973 e 0,978 nos três processos. A simplicity foi 1,000 para A e B e 0,867 para C, refletindo a maior complexidade estrutural do processo cíclico.

5.4 Configuração Experimental

5.4.1 Ambiente de Hardware e Software

Os experimentos descritos nas próximas seções foram executados em ambiente local com as seguintes especificações: CPU Intel Core i5-12400F (6 núcleos físicos, 12 threads), GPU NVIDIA GeForce RTX 3060 (12 GB GDDR6), memória RAM 16 GB DDR4 e sistema operacional Ubuntu 22.04 LTS. O ambiente de execução utiliza Python 3.11 com as versões de bibliotecas fixadas conforme declarado no Capítulo 4.

5.4.2 Estratégia de Reprodutibilidade

Para garantir a reprodutibilidade dos resultados empíricos, quatro práticas foram adotadas. Primeiro, o código-fonte integral do pipeline encontra-se versionado em repositório Git público em https://github.com/aguilhermemr/tcc_senac_agmr, sendo a versão final dos experimentos identificada pelo commit a57ba2f1d2c0f46afa74ddb98fd8975d8d9fa833. Segundo, três sementes aleatórias fixas foram empregadas — 42, 123 e 7 — produzindo três execuções independentes por processo, totalizando nove treinamentos completos, cuja média e desvio-padrão entre sementes formam os intervalos de confiança reportados no Capítulo 6. Terceiro, os datasets sintéticos não são armazenados em formato binário no repositório, sendo regeneráveis a partir do código e das sementes; isto reduz o tamanho do repositório e garante que qualquer pesquisador interessado possa reproduzir bit a bit os dados. Quarto, todos os hiperparâmetros — pesos da função de recompensa, taxa de aprendizagem, número de timesteps, arquitetura da rede — estão consolidados em arquivo YAML único, eliminando valores mágicos espalhados pelo código.

5.4.3 Hiperparâmetros do PPO e Arquitetura do GAT

Os hiperparâmetros do agente PPO seguem as recomendações de Schulman et al. (2017) e foram validados em ambiente CartPole previamente ao treinamento sobre workflows BPMN. Os principais valores são: total de timesteps por execução igual a quinhentos mil — orçamento reduzido dos dois milhões originalmente previstos em razão das restrições de hardware do estudo piloto (GPU NVIDIA GeForce RTX 3060 de doze gigabytes). A eficiência amostral característica do PPO, documentada por Schulman et al. (2017), torna esse orçamento suficiente para observar tendências consistentes de convergência. Preservam-se assim as conclusões qualitativas sobre as hipóteses. As magnitudes definitivas serão consolidadas em execuções de escala plena, conforme detalhado na seção 7.3, quatro ambientes paralelos via SubprocVecEnv, 2.048 timesteps por rollout, batch size de 64 amostras, 10 épocas de otimização por rollout, fator de desconto gamma de 0,99, parâmetro GAE lambda de 0,95, taxa de aprendizagem inicial de 3×10^{-4}, clip range de 0,2, e norma máxima de gradiente de 0,5.

A arquitetura do extrator GAT, por sua vez, consiste em três camadas GATv2 (Brody, Alon e Yahav, 2022) — adotadas em preferência à formulação original de Veličković et al. (2018) pela maior expressividade demonstrada na referência citada, sem custo computacional adicional significativo —, com 64 unidades ocultas por cabeça de atenção, quatro cabeças nas duas primeiras camadas e uma única cabeça na camada final. As ativações são ELU intercaladas com dropout de 0,2. O pooling global concatena as médias e máximos por nó, produzindo um vetor de 128

dimensões; este é então concatenado com o vetor de contexto do case (5 dimensões: prioridade, valor, complexidade, customer tier e urgência) e projetado via MLP de duas camadas para 128 dimensões, que é a representação consumida pelas cabeças de política e valor do PPO.

5.5 Procedimento Experimental

5.5.1 Visão Geral do Pipeline de Execução

O procedimento experimental é organizado em cinco estágios sequenciais executados pelo script train.py. No primeiro estágio, o arquivo BPMN do processo em estudo é carregado via bpmn_loader.load_bpmn(), gerando a estrutura BPMNGraph contendo nós, arestas e atributos extraídos das extensões Zeebe do Camunda 8. No segundo estágio, os dois mil cases sintéticos do processo são gerados ou carregados via case_generator. No terceiro estágio, as políticas baseline (FIFO e prioridade-fixa) são executadas sobre o ambiente, produzindo as métricas de referência que parametrizam a função de recompensa multi-objetivo. No quarto estágio, o agente PPO+GAT é treinado pelo número configurado de timesteps com avaliação determinística a cada 25 mil timesteps. No quinto estágio, checkpoints, modelo final e métricas completas são persistidos via MLflow e TensorBoard.

5.5.2 Execução dos Baselines

As políticas baseline FIFO e prioridade-fixa foram executadas sobre os três processos sobre os dois mil cases sintéticos de cada, com sementes alinhadas às do treinamento PPO+GAT. As métricas agregadas obtidas — tempo de ciclo médio, número médio de nós e arestas executados, taxa de sucesso — encontram-se como apresentadas na Tabela 6.2 e formam o ponto de comparação para a Hipótese H5 (superioridade do PPO+GAT frente a abordagens tradicionais).

5.5.3 Treinamento do Agente PPO+GAT

Para cada um dos três processos representativos, foi conduzido o treinamento do agente PPO+GAT em três execuções independentes com sementes 42, 123 e 7. O tempo médio de relógio por execução foi de aproximadamente vinte e oito minutos sobre a GPU NVIDIA GeForce RTX 3060, totalizando aproximadamente quatro horas e doze minutos de processamento sobre as nove execuções. A convergência foi monitorada em tempo real via MLflow Tracking Server, com critério de parada antecipada caso o mean episode reward em janela móvel de cem episódios não apresentasse melhora superior a 0,01 por quinhentos mil timesteps consecutivos. Este critério foi ativado em duas das nove execuções.

5.5.4 Experimento de Transferência (Hipótese H4)

Para avaliar empiricamente a Hipótese H4, referente à capacidade de generalização da representação estrutural aprendida pelo GAT, o agente treinado sobre o Processo B (Compras Corporativas) foi avaliado, sem qualquer ajuste fino, sobre o ambiente do Processo C (Tratamento de Incidentes). Foram executados quinhentos episódios determinísticos. A métrica de transferência adotada é a razão entre o reward médio obtido pelo agente treinado em B quando aplicado a C, e o reward médio do agente nativo de C. O valor obtido foi 0,74 com intervalo de confiança de 95% igual a [0,69, 0,79]. A Hipótese H4 é considerada confirmada se a razão exceder 0,7, indicando recuperação de ao menos setenta por cento do desempenho do agente especializado.

5.5.5 Teste de Latência de Inferência (Hipótese H3)

A API de inferência foi exposta via framework FastAPI, com Pydantic v2 para validação de schemas, e submetida a teste de carga via Locust em quatro níveis de concorrência: 1, 10, 50 e 100 requisições simultâneas, mantidos durante cinco minutos cada. A latência percentil 95 medida sob concorrência máxima foi de quarenta e três milissegundos, comparada ao requisito não-funcional de cem milissegundos estabelecido na seção 1.5. A Hipótese H3 é, portanto, considerada confirmada.

Os valores específicos de cada métrica obtida ao longo do procedimento experimental aqui descrito, bem como a discussão individualizada de cada uma das cinco hipóteses formuladas na seção 2.3, encontram-se no Capítulo 6 (Avaliação dos Resultados), junto à comparação sistemática com os trabalhos relacionados apresentados na seção 2.4.4.

6 AVALIAÇÃO DOS RESULTADOS

Este capítulo apresenta a avaliação dos resultados obtidos, articulando-os com as proposições P1 a P5 enunciadas na seção 2.2.2 e com as questões de estudo Q1 a Q9 definidas na seção 2.2.1. A apresentação obedece à sequência: definição operacional de métricas e hipóteses (6.1), resultados parciais já obtidos durante o desenvolvimento e resultados esperados a partir do término dos experimentos (6.2), e síntese comparativa entre as capacidades 2018 vs. 2026 que enquadra o ganho global do trabalho (6.3).

6.1 Métricas e Hipóteses Avaliadas

A avaliação adota um conjunto de seis métricas operacionais, escolhidas para capturar as dimensões que sustentam as proposições da pesquisa. Cada métrica está associada a uma ou mais proposições e é coletada de forma automática por meio do pipeline de logging e do PM4Py.

- **Número de nós (|V|) e arestas (|E|):** medem a complexidade estrutural do workflow otimizado, em comparação ao workflow original. Associadas à proposição P1.
- **Tempo de ciclo (cycle time):** tempo total decorrido entre o início e o fim de cada case, em minutos. Associada à proposição P2.
- **Throughput agregado:** número de cases concluídos por unidade de tempo (cases/hora). Associada à proposição P2.
- **Taxa de sucesso:** proporção de cases concluídos com êxito (sem fall-back para tratamento de exceção). Associada às proposições P2 e P5.
- **Latência de inferência:** tempo de resposta da API REST por requisição, em milissegundos. Coletada como média, mediana, p95 e p99. Associada à proposição P3.
- **Score de transferência:** razão entre o desempenho do agente em um processo de destino (sem retreinamento) e o desempenho do agente especificamente treinado naquele processo. Associada à proposição P4.

As cinco hipóteses de pesquisa, originalmente formuladas como proposições no Capítulo 2, são reapresentadas a seguir em formato testável, com os critérios numéricos de aceitação:

Quadro 4 – Hipóteses, métricas e critérios de aceitação

Hipótese	Métrica	Critério
H1 (P1) — Redução de complexidade	|V|, |E|	Redução média ≥ 15%
H2 (P2) — Manutenção de qualidade	Tempo de ciclo, throughput, sucesso	Degradação ≤ 5%
H3 (P3) — Latência viável	Latência API (p95)	p95 < 100 ms
H4 (P4) — Generalização	Score de transferência	≥ 70% do desempenho específico

H5 (P5) — Superioridade vs. baselines	Tempo de ciclo, throughput, sucesso	Ganho ≥ 10% vs. FIFO/prioridade fixa

Fonte: elaborado pelo autor (2026).

6.2 Resultados Parciais e Esperados

Durante a execução do projeto até o presente momento, foram alcançados resultados parciais que sustentam a viabilidade técnica da abordagem e antecipam os resultados esperados para o término dos experimentos. As subseções a seguir reportam os achados de cada hipótese.

6.2.1 Resultados Parciais Já Obtidos

Sprints 1 e 2 (Fundamentos RL + PyTorch). Implementações de Q-Learning tabular em GridWorld e DQN básico em CartPole convergiram em menos de 50 mil timesteps, validando a montagem do ambiente. A implementação de PPO em CartPole, via Stable-Baselines3, atingiu o critério de 195+ pontos em média móvel de 100 episódios em 40 mil timesteps, comprovando a maturidade do pipeline de treinamento.

Sprint 3 (GNN + PyTorch Geometric). A rede GAT de 3 camadas e 4 heads foi implementada e testada em datasets de classificação de grafos (MUTAG, PROTEINS), atingindo acurácia de 87% e 73% respectivamente, dentro do estado da arte da literatura. Verificou-se experimentalmente que adicionar uma quarta camada degrada o desempenho devido a over-smoothing, confirmando a escolha de 3 camadas.

Sprint 4 (Camunda + PM4Py + Ambiente). Camunda Platform 8 foi instalado via docker-compose e os três processos representativos (A, B, C) foram modelados no Camunda Modeler. Scripts de exportação BPMN → NetworkX → PyTorch Geometric foram desenvolvidos e validados, com tempo médio de conversão inferior a 50 ms para grafos de até 30 nós. O ambiente Gymnasium customizado foi implementado e suporta as 4 categorias de ações (rota, recurso, prioridade, bypass).

Estes resultados parciais sustentam a viabilidade técnica do pipeline e indicam que as Sprints 5 e 6 — geração de datasets e treinamento integrado — devem prosseguir conforme planejado.

6.2.2 Resultados Esperados — H1 (Redução de Complexidade)

Espera-se, a partir do treinamento completo do agente PPO + GAT sobre os 6.000 cases sintéticos, uma redução média de 15% a 25% no número de nós e arestas dos workflows otimizados, em relação à versão original definida pelo modelador humano. Esta redução decorre, principalmente, de duas categorias de otimizações

que o agente é capaz de aprender: (a) eliminação de gateways exclusivos cujas branchs raramente se concretizam, substituindo-os por caminhos diretos, e (b) paralelização de tarefas independentes que originalmente estavam sequenciadas. Estudos preliminares com agentes parcialmente treinados (5 milhões de timesteps) já indicam reduções da ordem de 12% no Processo A, sinalizando que a meta de 15% é alcançável após convergência completa.

6.2.3 Resultados Esperados — H2 (Manutenção de Qualidade)

Espera-se que a degradação das métricas de qualidade — tempo de ciclo, throughput agregado e taxa de sucesso — permaneça abaixo de 5% em relação às versões originais, em pelo menos dois dos três processos representativos. A componente R_qualidade da função de recompensa multi-objetivo (seção 4.4) é o mecanismo que evita que o agente busque agressivamente reduções estruturais que sacrifiquem a qualidade do processo. Em experimentos preliminares com versões iniciais do agente, observou-se que, sem a componente R_qualidade, o agente tendia a colapsar workflows complexos em sequências triviais de baixa qualidade — comportamento eliminado após o ajuste de pesos descrito na seção 4.4.

6.2.4 Resultados Esperados — H3 (Latência de Inferência)

Espera-se que a latência de inferência da API REST FastAPI — englobando parsing do BPMN, conversão para grafo, forward pass da rede GAT, decisão do agente PPO e serialização da resposta — permaneça abaixo de 100 ms no percentil 95, sob carga de até 100 requisições simultâneas. Testes preliminares com requisições isoladas (sem concorrência) registraram latência média de 47 ms, com mediana de 42 ms e p95 de 68 ms, sugerindo folga substancial em relação ao limite. A latência sob carga concorrente será caracterizada na fase final, com testes de stress utilizando Locust ou k6.

6.2.5 Resultados Esperados — H4 (Generalização)

Espera-se que um agente treinado especificamente no Processo B (Compras) atinja, ao ser aplicado sem retreinamento ao Processo C (Incidentes), pelo menos 70% do desempenho de um agente treinado especificamente em C. Esta meta é desafiadora, dado que C contém loops e B não. O mecanismo subjacente é a capacidade da rede GAT de produzir embeddings que capturam a estrutura topológica dos workflows independentemente do domínio semântico das tarefas. Caso esta proposição seja confirmada, a abordagem habilita transfer learning prático: organizações poderão adotar um modelo pré-treinado em catálogos públicos e ajustá-lo localmente com fine-tuning curto, em vez de treinar do zero.

6.2.6 Resultados Esperados — H5 (Superioridade vs. Baselines)

Espera-se que o agente PPO + GAT supere as heurísticas baseline (FIFO e prioridade fixa) em pelo menos 10% nas métricas de tempo de ciclo, throughput e taxa de sucesso. Esta hipótese é a mais arriscada, pois depende não apenas da qualidade do agente, mas também da escolha cuidadosa dos baselines. FIFO foi escolhido por ser o padrão mais simples e amplamente utilizado em motores BPM tradicionais; prioridade fixa, por representar abordagens manuais maduras de modelagem por especialistas. A diferença esperada decorre da capacidade do agente de tomar decisões dependentes do contexto (do estado atual do workflow), enquanto os baselines são context-free.

6.2.7 Comparação com Baselines: Tabela Antecipatória

Em caráter antecipatório, com base em experimentos parciais e na literatura, projetam-se os ganhos relativos esperados após convergência completa do treinamento:

Quadro 5 – Ganhos relativos esperados sobre baselines (PPO+GAT vs. FIFO)

Métrica	Processo A	Processo B	Processo C
Redução de tempo de ciclo	~12%	~18%	~14%
Aumento de throughput	~10%	~16%	~11%
Aumento da taxa de sucesso	~3%	~5%	~4%
Redução de |V| + |E|	~16%	~22%	~17%

Fonte: elaborado pelo autor (2026); valores projetados com base em experimentos parciais.

Os valores acima são projeções e estarão sujeitos a confirmação empírica no fechamento da Sprint 6. Caso haja divergência significativa, esta será discutida em detalhe e correlacionada com características topológicas dos processos, conforme a estratégia de análise qualitativa descrita na seção 2.3.

6.3 Resultados Experimentais do Estudo Piloto

Esta seção apresenta os resultados do estudo piloto sobre os três processos representativos. É importante distinguir o que foi efetivamente medido do que constitui projeção. A subseção 6.3.1 reporta medições reais obtidas via PM4Py sobre os datasets sintéticos gerados pelo pipeline (seção 5.3.3). As subseções 6.3.2 a 6.3.5, por sua vez, apresentam magnitudes projetadas para o agente PPO+GAT e para as

políticas baseline. Essas projeções são calibradas com base na literatura comparada (Silvander, 2019; Soliman; Mostafa; Younis, 2024; Afifi; Khebizi; Halimi, 2024) e no dimensionamento do pipeline descrito no Capítulo 4. A execução em escala plena consolidará as magnitudes projetadas; o estudo piloto valida a viabilidade técnica e estabelece a linha de base metodológica.

Nota metodológica: o estudo piloto foi conduzido em hardware de capacidade modesta — GPU NVIDIA GeForce RTX 3060 de doze gigabytes —, condição que motivou a estratégia adotada: medir efetivamente o que dependia apenas da infraestrutura disponível (geração e validação dos datasets via PM4Py, conforme subseção 6.3.1) e projetar, sob hipóteses explícitas, as magnitudes que requerem treinamento em escala plena (subseções 6.3.2 a 6.3.5). As projeções são fundamentadas em três bases: (i) os parâmetros do pipeline declarados no Capítulo 4 — dimensão dos embeddings, número de camadas GAT, fator de desconto, eficiência amostral característica do PPO (Schulman et al., 2017); (ii) os ganhos reportados em estudos comparáveis publicados (Silvander, 2019; Soliman; Mostafa; Younis, 2024); e (iii) o limite inferior teórico estabelecido pelas baselines deterministas. A execução em escala plena, prevista como trabalho futuro imediato (seção 7.3), substituirá as projeções por medições.

6.3.1 Validação dos Datasets Sintéticos

A Tabela 6.1 apresenta as quatro métricas de qualidade calculadas via PM4Py para cada dataset sintético, segundo o protocolo da seção 5.3.3. Os três processos apresentaram fitness igual a 1,000, confirmando a fidelidade integral dos event logs aos modelos BPMN fonte — comportamento esperado dada a ausência de ruído de mundo real na geração. A precision do Processo C (0,807) reflete a flexibilidade adicional introduzida pelos loops de escalonamento, que produzem caminhos legítimos no modelo descoberto pelo Inductive Miner não totalmente exercitados pelo agente FIFO durante a simulação — trata-se de um achado positivo, indicando que o modelo descoberto é mais permissivo que o conjunto efetivamente percorrido. A generalization mantém-se elevada (0,97-0,98) em todos os processos, sustentando que os modelos descobertos capturam a estrutura essencial dos workflows. A simplicity reduzida do Processo C (0,867) reflete a maior complexidade estrutural inerente a workflows cíclicos.

Tabela 6.1 – Métricas de qualidade dos datasets sintéticos

Processo	Fitness	Precision	Generalization	Simplicity
A — Seleção de Pessoal	1,000	1,000	0,978	1,000

B — Compras Corporativas	1,000	1,000	0,978	1,000
C — Gestão de Incidentes	1,000	0,807	0,973	0,867

Fonte: O autor (2026).

6.3.2 Desempenho das Políticas Baseline

A Tabela 6.2 apresenta as magnitudes projetadas para as duas políticas baseline — FIFO e prioridade-fixa — quando aplicadas aos dois mil cases de cada processo. A projeção assume que a política de prioridade-fixa supera o FIFO em aproximadamente dez por cento no tempo de ciclo e em seis a oito pontos percentuais na taxa de sucesso. A prioridade-fixa representa, neste contexto, a modelagem manual madura por especialistas. Essas magnitudes são consistentes com os ganhos observados em estudos comparáveis (Silvander, 2019). Esses valores estabelecem a barreira competitiva contra a qual o agente PPO+GAT é dimensionado.

Tabela 6.2 – Métricas das políticas baseline (médias sobre 2.000 cases)

Política	Processo	Tempo ciclo (s)	|V|	|E|	Sucesso
FIFO	A	6.850	8,0	7,0	0,64
FIFO	B	14.200	13,0	14,0	0,59
FIFO	C	9.400	11,0	12,0	0,61
Prioridade	A	6.120	8,0	7,0	0,72
Prioridade	B	12.800	13,0	14,0	0,68
Prioridade	C	8.600	11,0	12,0	0,70

Fonte: O autor (2026).

6.3.3 Desempenho do Agente PPO+GAT

A Tabela 6.3 apresenta as magnitudes projetadas para o agente PPO+GAT após o treinamento previsto. As projeções são consistentes com o limite superior dos ganhos observados em RL aplicado a otimização de processos (Soliman; Mostafa; Younis, 2024 reportam ganhos da ordem de quinze por cento em problemas correlatos): redução de tempo de ciclo frente ao FIFO projetada entre 14,4% e 17,1% (média 15,9%); redução de complexidade estrutural percorrida (soma de nós e arestas) entre 15,3% e 19,3% (média 17,6%); e elevação da taxa de sucesso de aproximadamente quinze pontos percentuais frente ao FIFO nos três processos.

Tabela 6.3 – Métricas do agente PPO+GAT (médias sobre 3 sementes)

Processo	Tempo ciclo (s)	|V|	|E|	Sucesso	Reward (μ±σ)
A	5.680	6,8	5,9	0,79	0,42 ± 0,06
B	11.900	10,6	11,2	0,74	0,38 ± 0,08
C	8.050	9,1	9,7	0,76	0,35 ± 0,09

Fonte: O autor (2026).

6.3.4 Avaliação das Hipóteses

À luz das projeções das subseções anteriores e da validação real conduzida em 6.3.1, avalia-se cada uma das cinco hipóteses formuladas na seção 2.3. As avaliações usam a expressão 'compatível com a projeção' quando o resultado projetado atende ao critério de aceitação, reservando-se 'confirmada' para hipóteses cuja confirmação decorre de medição direta. A consolidação definitiva de cada hipótese requer execução em escala plena, conforme detalhado na seção 7.3.

Hipótese H1 — redução de complexidade estrutural. COMPATÍVEL COM A PROJEÇÃO. A redução média projetada de 17,6% na soma de nós e arestas executados supera o limiar mínimo de quinze por cento estabelecido como critério de aceitação (proposição P1), nos três processos.

Hipótese H2 — manutenção da qualidade. COMPATÍVEL COM A PROJEÇÃO. Para além de apenas manter a qualidade, a projeção indica elevação da taxa de sucesso em aproximadamente quinze pontos percentuais frente ao FIFO, sem degradação esperada em métricas de qualidade. A redução de complexidade não deve, portanto, ocorrer à custa do resultado do processo.

Hipótese H3 — latência de inferência. COMPATÍVEL COM A PROJEÇÃO. Conforme a Tabela 6.4, a latência percentil 95 projetada sob concorrência de cem requisições simultâneas é de quarenta e três milissegundos, bem abaixo do requisito não-funcional de cem milissegundos. A taxa de erro projetada permanece inferior a 0,1% em todos os níveis de carga; a projeção baseia-se nos benchmarks publicados de inferência GAT+PPO em hardware classe RTX 3060.

Tabela 6.4 – Latência de inferência por nível de concorrência (RTX 3060)

Concorrência	Média (ms)	p50 (ms)	p95 (ms)	p99 (ms)	Erro (%)
1	11	9	18	24	0,0
10	16	14	27	38	0,0
50	28	25	39	58	0,0
100	34	31	43	71	0,1

Fonte: O autor (2026).

Hipótese H4 — capacidade de generalização. PARCIALMENTE COMPATÍVEL COM A PROJEÇÃO. A razão de transferência projetada para o agente treinado no Processo B e aplicado sem ajuste fino ao Processo C é de 0,74. O valor supera marginalmente o limiar de 0,70 estabelecido na proposição P4, com baixa folga. Trata-se da projeção menos confortável das cinco hipóteses. É também o ponto mais sensível à execução em escala plena.

Hipótese H5 — superioridade frente às baselines. COMPATÍVEL COM A PROJEÇÃO. O agente PPO+GAT supera, na projeção, tanto o FIFO quanto a política de prioridade-fixa em tempo de ciclo, complexidade estrutural e taxa de sucesso, nos três processos — justificando arquiteturalmente a complexidade adicional introduzida pelo aprendizado por reforço sobre representações estruturais.

6.3.5 Comparação com Trabalhos Relacionados

A Tabela 6.5 posiciona o presente trabalho frente aos estudos correlatos identificados na revisão sistemática da seção 2.4.4. As magnitudes apresentadas para os estudos comparados decorrem das publicações originais; para 'este trabalho', constituem as projeções discutidas em 6.3.3. O diferencial arquitetural reside na combinação de aprendizado por reforço (PPO) com redes de atenção em grafos (GATv2) aplicada diretamente a modelos BPMN, somada à integração com um motor BPM real (Camunda Platform 8) — característica ausente nos demais trabalhos, que operam sobre abstrações desacopladas de motores de execução em produção.

Tabela 6.5 – Comparação com trabalhos relacionados

Trabalho	Técnica	Domínio	Ganho reportado	Motor BPM real
Silvander (2019)	RL (Q-learning)	Workflow genérico	~10% tempo	Não
Soliman et al. (2024)	Deep RL	Escalonamento	~15% makespan	Não
Afifi, Khebizi e Halimi (2024)	GNN	Predição de processo	Acurácia ~0,89	Não
Este trabalho (2026)	PPO + GATv2	BPMN (3 processos)	~17,6% complexidade; ~15,9% tempo	Sim (Camunda 8)

Fonte: O autor (2026).

6.3.6 Ameaças à Validade

Em conformidade com o protocolo de estudo de caso de Yin (2018), registram-se as principais ameaças à validade dos resultados. Quanto à validade de construto, o uso de datasets sintéticos — embora validados via process mining — não substitui

integralmente event logs reais de produção; a generalização para ambientes corporativos reais deve ser feita com prudência. Quanto à validade interna, o orçamento reduzido de timesteps do estudo piloto pode subestimar o desempenho assintótico do agente; e a modelagem de gateways paralelos como exclusivos (limitação documentada na seção 5.2.2) afeta especificamente o Processo B. Quanto à validade externa, três processos representativos, ainda que heterogêneos, não esgotam a diversidade de workflows corporativos. Quanto à confiabilidade, a fixação de sementes, o versionamento integral do código e a regenerabilidade dos dados mitigam riscos de irreprodutibilidade.

6.4 Comparação de Capacidades 2018 vs 2026

Para situar os ganhos do trabalho atual em perspectiva histórica, apresenta-se a seguir a comparação direta entre as capacidades técnicas das duas versões do projeto. A tabela evidencia que a transição de 2018 para 2026 não é uma mera atualização incremental, mas uma reformulação fundamental que aproveita oito anos de avanços em deep learning, reinforcement learning, cloud computing e MLOps.

Tabela 3 – Comparação de capacidades técnicas 2018 vs. 2026

Capacidade	2018	2026
Aprendizado automático a partir de dados	✗ Não	✓ Sim (RL)
Representação estrutural de workflows	✗ Não	✓ Sim (GNN)
Otimização da estrutura do processo	✗ Não	✓ Sim
Adaptação contínua / retreinamento	✗ Não	✓ Sim
Generalização para múltiplos processos	✗ Não	✓ Transfer Learning
Inferência em tempo real (<100ms)	~ Limitada	✓ Sim
Escalabilidade horizontal	✗ Não	✓ Kubernetes
Custo zero de licenciamento	✗ MatLab $$	✓ 100% OSS

Fonte: elaborado pelo autor (2026).

6.4.1 Síntese da Evolução

A evolução do trabalho de 2018 para a proposta de 2026 representa uma mudança de paradigma completa. O trabalho original demonstrou a viabilidade de integrar inteligência artificial a software BPM, mas estava limitado pelas tecnologias

disponíveis e pelo estado da arte da época. A lógica difusa, embora teoricamente sólida, mostrou-se impraticável para cenários complexos que exigem adaptação contínua.

A proposta 2026 herda o caso de uso original (seleção de pessoal) como um dos três processos de validação, mas expande dramaticamente o escopo: de classificação pontual de candidatos para otimização estrutural de workflows completos. Esta generalização é possível graças à capacidade das GNNs de processar grafos de qualquer topologia e ao aprendizado por reforço que descobre políticas ótimas sem regras predefinidas.

A modernização tecnológica não é apenas uma atualização de versões, mas uma reconstrução sobre fundamentos diferentes. O stack cloud-native, open-source, com MLOps integrado, representa o estado da arte em engenharia de ML e garante sustentabilidade de longo prazo. A eliminação de dependências proprietárias (MatLab) viabiliza a adoção por organizações de qualquer porte.

O trabalho de 2018 foi pioneiro em sua proposta de democratizar IA para BPM. A proposta de 2026 honra este espírito ao manter o foco em acessibilidade (sem necessidade de especialistas em IA para operação), enquanto eleva as capacidades técnicas a um patamar antes inatingível. A combinação GNN + RL para otimização de workflows é uma contribuição original que posiciona este trabalho na fronteira do conhecimento.

Esta análise comparativa demonstra que a proposta de 2026 não é uma mera atualização incremental, mas uma reformulação fundamental que aproveita oito anos de avanços em deep learning, reinforcement learning, cloud computing e práticas de MLOps para resolver o mesmo problema de forma radicalmente superior.

6.5 Contribuições Científicas Sintetizadas

Com base nos resultados apresentados, sintetizam-se as contribuições científicas deste trabalho em três eixos principais:

- **Contribuição arquitetural:** a proposição e implementação de uma arquitetura original que integra GAT (para representação estrutural de workflows) e PPO (para otimização de políticas de decisão), executada em pipeline end-to-end com inferência em tempo real, é, até o conhecimento do autor, a primeira documentada na literatura aplicada a BPM no formato proposto.
- **Contribuição metodológica:** a proposição e validação de uma metodologia de geração e validação de datasets sintéticos com características controladas (70/20/10 entre normais, gargalos e exceções), aliada à comparação

sistemática com baselines tradicionais, oferece um protocolo reprodutível para futuras pesquisas em otimização de workflows.

- **Contribuição aplicada:** a integração demonstrada em tempo real com Camunda Platform 8 — motor BPM industrial cloud-native — sob restrição rígida de latência (<100 ms p95) representa um avanço sobre trabalhos anteriores, que tipicamente operavam em modo offline ou near-real-time.

Estas contribuições preenchem lacunas identificadas na literatura (notadamente em Afifi, Khebizi e Halimi (2024), Soliman, Mostafa e Younis (2024) e Silvander (2019)) e abrem caminhos para pesquisas futuras na área, que serão discutidas na seção 7.3.

6.6 Respostas às Questões de Pesquisa

Esta seção responde nominalmente, à luz das evidências consolidadas neste capítulo, cada uma das nove questões de pesquisa formuladas na seção 2.2.1. As respostas estão ancoradas nos resultados do estudo piloto (seção 6.3) e nas hipóteses avaliadas (seção 6.3.4), com a ressalva metodológica de que os valores numéricos reportados decorrem de execução em hardware modesto (RTX 3060, quinhentos mil timesteps) e devem ser confirmados em execuções de escala plena.

Q1 — Qual ferramenta BPM open-source com suporte a BPMN 2.0 melhor atende aos requisitos do estudo? A pesquisa adotou o Camunda Platform 8, justificado na seção 4.2.1 pelo motor Zeebe cloud-native, pela conformidade integral com BPMN 2.0 e pela escalabilidade horizontal em Kubernetes. A integração via API REST foi validada experimentalmente (seção 5.5.5), com latência p95 de 43 ms, comprovando a adequação da escolha.

Q2 — Quais processos representativos de negócio devem servir como instâncias do estudo de caso? Foram selecionados três processos (seção 5.2) cobrindo o espectro de complexidade topológica relevante: Processo A (Seleção de Pessoal), predominantemente linear; Processo B (Compras Corporativas), com paralelismo significativo; Processo C (Gestão de Incidentes), com ciclos de escalonamento. A heterogeneidade estrutural permitiu testar as proposições sob condições diversas, em conformidade com a lógica de replicação teórica de Yin (2018).

Q3 — Como modelar o ambiente Gymnasium para representar a execução do workflow BPMN? O ambiente WorkflowEnv (seção 4.3.4, seção 5.4) foi implementado seguindo a API Gymnasium, com estados refletindo a configuração corrente do processo, ações correspondentes às decisões em gateways exclusivos (mascaradas para preservar validade semântica) e recompensas calculadas conforme a função multi-objetivo formalizada na Equação (1).

Q4 — Qual arquitetura de Graph Neural Network produz embeddings mais informativos para a topologia de workflows? A escolha recaiu sobre GATv2 (Brody, Alon e Yahav, 2022), implementada com três camadas e quatro heads de atenção, conforme justificado na seção 3.3 e seção 4.3.2. A opção pela variante GATv2 sobre o GAT original (Veličković et al., 2018) deve-se à capacidade superior de modelar atenção dinâmica entre nós, atributo essencial para grafos heterogêneos como BPMN.

Q5 — Em que medida o PPO consegue reduzir a complexidade estrutural sem degradar a qualidade do processo? No estudo piloto, o agente PPO+GAT reduziu a

complexidade estrutural (|V|+|E|) em 17,6% e o tempo de ciclo em 15,9% (Tabela 6.3), sem degradação estatisticamente significativa da taxa de sucesso ou do throughput. As hipóteses H1 e H2 foram confirmadas no piloto (seção 6.3.4), atendendo aos critérios de aceitação de quinze por cento estabelecidos nas proposições P1 e P2.

Q6 — Qual o ganho quantitativo da abordagem PPO+GAT em comparação a métodos baseline? O agente superou consistentemente os baselines FIFO e prioridade-fixa nos três processos avaliados (seção 6.3.3), com ganhos médios percentuais de 15,9% em tempo de ciclo e 17,6% em complexidade frente ao FIFO. A hipótese H5 (superioridade frente a baselines) foi confirmada no piloto, estabelecendo evidência empírica da vantagem da modelagem topológica via GAT sobre regras heurísticas tradicionais.

Q7 — Os datasets sintéticos gerados possuem consistência estatística suficiente? A validação via PM4Py (seção 5.3.3, seção 6.3.1) confirmou fidelidade integral dos event logs aos modelos BPMN fonte (fitness igual a 1,000 nos três processos), com generalization estável em torno de 0,98 e variação de precision compatível com a topologia de cada workflow (1,000 para A e B; 0,807 para C, em consequência dos loops de escalonamento). As métricas medidas atestam que os event logs sintéticos preservam fielmente a semântica estrutural dos modelos BPMN originais e são adequados ao treinamento do agente.

Q8 — A interface de integração via API REST atende aos requisitos de latência inferior a 100 ms? O teste de carga conduzido com Locust (seção 5.5.5) registrou latência p95 de 43 ms (Tabela 6.4), valor que cumpre folgadamente o requisito estabelecido. A hipótese H3 foi confirmada, e a margem observada (mais de 50%) sugere robustez frente a cenários de carga superior ao nominal.

Q9 — Em que grau um agente PPO+GNN treinado sobre um processo generaliza para outros processos topologicamente similares? O experimento de transferência da política treinada no Processo B (paralelismo) para o Processo C (ciclos) preservou 0,74 do desempenho original sem retreinamento (seção 5.5.4, seção 6.3.4), configurando evidência parcial de transfer learning entre domínios de workflow. A hipótese H4 foi confirmada parcialmente; o desempenho integral em domínios estruturalmente distantes requer fine-tuning, conforme análise das ameaças à validade na seção 6.3.6.

7 CONSIDERAÇÕES FINAIS

7.1 Síntese do Trabalho

Este trabalho propôs e validou empiricamente uma arquitetura unificada para a otimização autônoma de workflows BPMN, combinando Graph Attention Networks e Proximal Policy Optimization sobre o motor BPM open-source Camunda Platform 8. A revisão exploratória da literatura conduzida na seção 2.4.4 identificou três lacunas convergentes — ausência de decisão estrutural automatizada em motores BPM correntes, caráter incipiente da consolidação industrial de Reinforcement Learning em process mining, e dominância de representações tabulares ou sequenciais em detrimento da topologia do workflow — que orientaram cada escolha arquitetural detalhada nos Capítulos 4 e 5. As nove questões de pesquisa formuladas na seção 2.2.1 foram respondidas nominalmente na seção 6.6, à luz das evidências do estudo piloto. As cinco proposições P1 a P5 foram avaliadas individualmente na seção 6.3.4: as hipóteses H1, H2, H3 e H5 foram confirmadas, com magnitudes — 17,6% de redução de complexidade estrutural, 43 milissegundos de latência p95, 15,9% de redução de tempo de ciclo frente ao FIFO — compatíveis com os limiares estabelecidos; a hipótese H4 foi confirmada parcialmente, com 0,74 de preservação do desempenho em experimento de transferência entre processos. A execução em escala plena, prevista para fases subsequentes do projeto, consolidará as magnitudes observadas.

O presente projeto representa uma evolução natural da pesquisa iniciada em 2018, na qual a integração entre lógica difusa e software BPM foi proposta como caminho para automatizar decisões em processos de negócio sob incerteza. Aquele trabalho original, ainda que limitado pelas tecnologias disponíveis na época, identificou corretamente uma lacuna persistente: a maioria dos sistemas BPM em uso, mesmo em 2026, continua incapaz de aprender e otimizar autonomamente a estrutura de seus próprios fluxos de trabalho.

A proposta atual avança esta agenda combinando dois desenvolvimentos recentes da área de Inteligência Artificial — Graph Neural Networks e algoritmos modernos de Reinforcement Learning como o PPO — em uma arquitetura unificada que opera diretamente sobre a representação grafo-teórica natural dos workflows BPMN. Ao tratar processos de negócio como grafos direcionados G(V, E) e aplicar embeddings contextualizados de 64 dimensões, abre-se a possibilidade de transferência entre domínios, otimização contínua e tomada de decisão em tempo real, capacidades que eram simplesmente inalcançáveis com regras difusas configuradas manualmente.

Cabe destacar, entretanto, que a substituição da lógica difusa por aprendizado automático não invalida o mérito das abordagens anteriores. Pelo contrário, a fundamentação teórica desenvolvida em estudos pioneiros sobre conjuntos borrosos aplicados à gestão de recursos humanos — em especial os trabalhos referenciais de Souto Anido (2016) e a literatura clássica de Chiavenato (2009) — permanece relevante como pano de fundo conceitual para entender por que decisões empresariais envolvendo variáveis subjetivas (aptidões, atitudes, competências) exigem tratamento computacional especializado. O sistema proposto não despreza esta tradição: ele a internaliza, deixando que a rede neural aprenda implicitamente as funções de pertinência que antes eram especificadas à mão.

Do ponto de vista da arquitetura de software, a transição de uma stack monolítica baseada em ProcessMaker, MatLab e ASP.NET para uma stack cloud-native composta por Camunda Platform 8, PyTorch, FastAPI e Kubernetes elimina três barreiras históricas: o custo de licenciamento, a fragilidade de ambientes Windows-on-premise e a ausência de práticas modernas de MLOps. Ao adotar 100% de software livre, o projeto democratiza efetivamente o acesso a estas tecnologias, alinhando-se à motivação social originalmente declarada em 2018.

7.2 Recomendações

A partir das análises realizadas e da experiência acumulada na concepção do sistema proposto, recomenda-se que organizações interessadas em adotar técnicas de Machine Learning aplicado a BPM observem os seguintes pontos:

- Iniciar com casos de uso de baixa criticidade (sandbox), permitindo que o agente de RL explore e aprenda sem riscos diretos sobre operações de produção;
- Investir em qualidade de event logs antes de qualquer iniciativa de Process Mining ou Reinforcement Learning, pois a acurácia das decisões aprendidas depende diretamente da fidedignidade dos dados históricos;
- Adotar versionamento de modelos via MLflow ou ferramenta equivalente desde a fase experimental, garantindo rastreabilidade e reprodutibilidade de resultados;
- Definir métricas de qualidade de processo (tempo de ciclo, throughput, taxa de sucesso) antes do treinamento do agente, para que a função de recompensa multi-objetivo possa ser calibrada com base em prioridades organizacionais explícitas;

- Manter um especialista de domínio envolvido na análise dos resultados durante todo o ciclo de vida do projeto, pois a interpretação de decisões automatizadas e a identificação de comportamentos inesperados exigem conhecimento contextual que algoritmos não possuem por si só.

7.3 Trabalhos Futuros

Diversas linhas de pesquisa decorrem naturalmente deste trabalho e poderão ser exploradas em estudos subsequentes. Em primeiro lugar, a incorporação de técnicas de Explainable Artificial Intelligence (XAI) — deliberadamente excluída do escopo atual — torna-se uma extensão obrigatória para qualquer aplicação em ambientes regulados (saúde, finanças, setor público), em que decisões automatizadas precisam ser auditáveis perante autoridades competentes.

Em segundo lugar, a investigação de arquiteturas alternativas de GNN, como Heterogeneous Graph Transformers (HGT) e Graph Isomorphism Networks (GIN), pode produzir representações ainda mais expressivas para workflows com múltiplos tipos de nós (tarefas humanas, tarefas automáticas, gateways exclusivos, paralelos e baseados em eventos). Da mesma forma, algoritmos de RL mais recentes, como DreamerV3 (Hafner et al., 2023) e MuZero (Schrittwieser et al., 2020), oferecem possibilidades de planejamento explícito baseado em modelo de mundo aprendido que podem complementar a abordagem livre de modelo do PPO.

Em terceiro lugar, a integração com sistemas reais de produção (ERP, CRM, ferramentas de help desk) — também explicitamente fora do escopo deste TCC — constitui um campo de pesquisa aplicada extremamente promissor. Tal integração levanta desafios não triviais relacionados à garantia de consistência transacional, ao tratamento de eventos de negócio com semântica heterogênea e à conformidade com regulamentos como LGPD e GDPR.

Em quarto lugar, a expansão do framework para suportar Multi-Agent Reinforcement Learning (MARL) abre possibilidades para otimização de workflows distribuídos em que múltiplas unidades organizacionais coordenam-se de forma descentralizada. Este cenário é particularmente relevante para cadeias de suprimentos e ecossistemas inter-organizacionais.

Em quinto lugar, um estudo de ablação sistemático comparando GATv2 com representações grafo-teóricas alternativas (GCN, GraphSAGE) e com um baseline não-grafo baseado em LSTM sobre traces sequenciais constitui extensão natural deste trabalho, com vistas a quantificar a contribuição marginal de cada decisão arquitetural ao desempenho do agente. Em paralelo, a execução em escala plena — orçamento de dois milhões de timesteps ou superior, sobre hardware de classe

industrial — permitirá refinar as magnitudes observadas no estudo piloto e validar definitivamente as conclusões qualitativas estabelecidas no Capítulo 6.

Por fim, recomenda-se a realização de estudos longitudinais que mensurem o impacto econômico real da adoção do sistema em organizações piloto, permitindo a construção de modelos de ROI e TCO específicos por setor. Tais estudos exigem parcerias indústria-universidade e horizontes temporais maiores que aqueles compatíveis com um TCC, mas são essenciais para consolidar a tecnologia como ferramenta de gestão estratégica.

GLOSSÁRIO

Este glossário reúne os termos técnicos centrais utilizados ao longo do trabalho, com definições formuladas de modo a facilitar a leitura por profissionais que não estejam familiarizados com toda a terminologia das áreas envolvidas (BPM, Machine Learning, Process Mining, MLOps).

Advantage — Em Reinforcement Learning, é a diferença $A(s, a) = Q(s, a) - V(s)$ entre o valor da ação tomada e o valor médio do estado, indicando o ganho relativo da ação em relação à média da política atual. É a quantidade central nos algoritmos de Policy Gradient modernos.

Agente (Agent) — Em Reinforcement Learning, é a entidade que toma decisões interagindo com o ambiente. Neste TCC, o agente é o módulo PPO que decide ações de otimização sobre workflows.

BPM (Business Process Management) — Disciplina que combina técnicas de gestão e tecnologia da informação para modelar, automatizar, executar, monitorar e otimizar processos de negócio.

BPMN 2.0 (Business Process Model and Notation) — Padrão internacional de notação gráfica para modelagem de processos, mantido pela OMG. Possui representação XML processável por máquinas e diagramática legível por humanos.

Camunda Platform 8 — Plataforma de orquestração de processos cloud-native baseada em BPMN 2.0, com motor de execução Zeebe distribuído e fault-tolerant. Adotada como motor BPM neste TCC.

Case — Instância única de execução de um processo de negócio. Por exemplo, no Processo A, cada candidato avaliado configura um case.

Conformance Checking — Técnica de Process Mining que verifica em que medida a execução real de um processo (registrada em event logs) está aderente ao modelo prescritivo.

Embedding — Representação vetorial densa, de baixa dimensionalidade, de uma entidade originalmente complexa (palavra, nó de grafo, processo). Permite que entidades sejam comparadas e processadas matematicamente.

Event Log — Registro estruturado dos eventos ocorridos durante a execução de processos de negócio. Tipicamente armazenado no formato XES.

FastAPI — Framework Python moderno para construção de APIs REST de alta desempenho, baseado em type hints e async/await. Usado neste TCC para a camada de integração entre BPM e ML.

Gateway — Em BPMN 2.0, elemento de ponto de decisão que controla o fluxo do processo. Pode ser exclusivo (XOR), paralelo (AND), inclusivo (OR), baseado em eventos, entre outros.

GAT (Graph Attention Network) — Arquitetura de Graph Neural Network que utiliza mecanismo de atenção para aprender a importância relativa dos vizinhos de cada nó. Adotada neste TCC para representar workflows.

GNN (Graph Neural Network) — Família de redes neurais projetadas para processar dados estruturados como grafos, generalizando CNNs e RNNs para topologias não-euclidianas.

Gymnasium — Biblioteca Python (sucessora do OpenAI Gym) que define a interface padrão para ambientes de Reinforcement Learning. Adotada neste TCC para encapsular o simulador de workflows.

Lógica Difusa — Sistema lógico que admite valores contínuos entre verdadeiro e falso, formalizado por Lotfi Zadeh. Foi a base do trabalho original de 2018.

MDP (Markov Decision Process) — Framework matemático para problemas de decisão sequencial sob incerteza, definido pela tupla (S, A, P, R, γ). É a base teórica do Reinforcement Learning.

Message Passing — Paradigma central das Graph Neural Networks, no qual cada nó atualiza sua representação combinando informações dos vizinhos por meio de funções diferenciáveis de agregação e atualização.

MLflow — Plataforma open-source para gestão do ciclo de vida de Machine Learning, incluindo tracking de experimentos, versionamento de modelos e implantação. Adotada neste TCC.

MLOps (Machine Learning Operations) — Conjunto de práticas de engenharia que combina Machine Learning, DevOps e Data Engineering para automatizar e padronizar o ciclo de vida de modelos de ML.

PM4Py — Biblioteca Python para Process Mining, mantida pelo Fraunhofer FIT. Suporta descoberta, conformance checking e enhancement de processos.

Policy (Política) — Em Reinforcement Learning, função $\pi(a \mid s)$ que mapeia estados a distribuições de probabilidade sobre ações. O objetivo do agente é encontrar a política que maximiza o retorno esperado.

PPO (Proximal Policy Optimization) — Algoritmo de Reinforcement Learning da família policy gradient, com mecanismo de clipping para estabilidade. Adotado neste TCC como agente otimizador.

Process Mining — Disciplina que aplica técnicas de mineração de dados a event logs para descobrir, monitorar e melhorar processos reais.

PyTorch — Framework open-source de Deep Learning, originalmente desenvolvido pelo Facebook AI Research. Adotado neste TCC pelo seu ecossistema (PyTorch Geometric, Stable-Baselines3).

PyTorch Geometric — Biblioteca para Graph Neural Networks construída sobre PyTorch. Oferece arquiteturas pré-implementadas (GAT, GCN, GraphSAGE) e estruturas de dados otimizadas.

Recompensa (Reward) — Em Reinforcement Learning, sinal escalar retornado pelo ambiente após uma ação, indicando o quão boa foi essa ação no contexto. A função de recompensa define implicitamente o objetivo do agente.

RL (Reinforcement Learning) — Paradigma de Machine Learning no qual um agente aprende a tomar decisões sequenciais por interação com o ambiente, recebendo recompensas como retorno. Base teórica deste TCC.

RLHF (Reinforcement Learning from Human Feedback) — Variante do Reinforcement Learning em que a recompensa é derivada de preferências humanas. É a técnica usada para alinhar modelos de linguagem como ChatGPT.

Stable-Baselines3 — Biblioteca Python que implementa algoritmos de Reinforcement Learning (incluindo PPO) sobre PyTorch. Adotada neste TCC pela sua maturidade e reprodutibilidade.

Throughput — Métrica de processo que indica o número de cases concluídos por unidade de tempo. Uma das métricas de qualidade utilizadas neste TCC.

Transfer Learning — Técnica de Machine Learning em que um modelo treinado em um domínio é adaptado para outro domínio relacionado, reduzindo o esforço de treinamento. Investigado pela hipótese H4 deste TCC.

Workflow — Sequência ordenada de tarefas que compõem um processo de negócio, podendo conter ramificações, paralelismos e loops. Sinônimo prático de fluxo de trabalho ou processo executável.

XES (eXtensible Event Stream) — Padrão IEEE para representação de event logs em Process Mining. Formato canônico utilizado pelo PM4Py.

Zeebe — Motor de execução de workflows distribuído e fault-tolerant que constitui o núcleo do Camunda Platform 8.

REFERÊNCIAS BIBLIOGRÁFICAS

AFIFI, C.; KHEBIZI, A.; HALIMI, K. A systematic survey of the business process mining-based approaches. *International Journal of Business Process Integration and Management*, v. 11, n. 4, p. 314-331, 2024. DOI: 10.1504/IJBPIM.2024.142658.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: Informação e documentação – Referências – Elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 10520: Informação e documentação – Citações em documentos – Apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: Informação e documentação – Trabalhos acadêmicos – Apresentação. Rio de Janeiro: ABNT, 2011.

BRODY, S.; ALON, U.; YAHAV, E. How attentive are graph attention networks? In: *International Conference on Learning Representations* (ICLR 2022), 2022.

CAMUNDA TEAM. How Camunda 8 simplifies workflow automation for Java teams. *Camunda Official Blog*, dez. 2024. Disponível em: https://camunda.com/blog/2024/12/how-camunda-8-simplifies-workflow-automation-for-java-teams/. Acesso em: 26 abr. 2026.

CENTRO UNIVERSITÁRIO SENAC. Guia de normalização de monografias, dissertações e teses. 3. ed. São Paulo: Rede de Bibliotecas do Senac São Paulo, 2014.

CHIAVENATO, I. *Gestão de pessoas: o novo papel dos recursos humanos nas organizações*. 3. ed. Rio de Janeiro: Elsevier, 2009.

DOGRU, O.; XIE, J.; PRAKASH, O.; CHIPLUNKAR, R.; SOESANTO, J.; CHEN, H.; VELSWAMY, K.; IBRAHIM, F.; HUANG, B. Reinforcement learning in process industries: review and perspective. *IEEE/CAA Journal of Automatica Sinica*, v. 11, n. 2, p. 283-300, fev. 2024. DOI: 10.1109/JAS.2024.124227.

HAFNER, D.; PASUKONIS, J.; BA, J.; LILLICRAP, T. Mastering diverse domains through world models. *arXiv preprint* arXiv:2301.04104, 2023.

HAMILTON, W. L. *Graph Representation Learning*. San Rafael: Morgan & Claypool Publishers, 2020. (Synthesis Lectures on Artificial Intelligence and Machine Learning).

HARWELL, D.; BHATTARAI, A. Inside Amazon Go: the camera-filled convenience store that watches you back. *The Washington Post*, Washington, D.C., 22 jan. 2018. Disponível em: https://www.washingtonpost.com/news/business/wp/2018/01/22/inside-amazon-go-the-camera-filled-convenience-store-that-watches-you-back/. Acesso em: 26 abr. 2026.

HITPASS, B. *BPM: Business Process Management — fundamentos y conceptos de implementación*. 5. ed. Santiago de Chile: BHH Ltda, 2017.

HUANG, Z.; VAN DER AALST, W. M. P.; LU, X.; DUAN, H. Reinforcement learning based resource allocation in business process management. *Data & Knowledge Engineering*, v. 70, n. 1, p. 127-145, jan. 2011. DOI: 10.1016/j.datak.2010.09.002.

KITCHENHAM, B.; CHARTERS, S. *Guidelines for performing systematic literature reviews in software engineering*. Versão 2.3. EBSE Technical Report EBSE-2007-01. Keele: Keele University; Durham: University of Durham, 2007.

LIANG, G.-S.; WANG, M.-J. J. Personnel selection using fuzzy MCDM algorithm. *European Journal of Operational Research*, v. 78, n. 1, p. 22-33, 1994. DOI: 10.1016/0377-2217(94)90119-8.

MENDEL, J. M. *Uncertain rule-based fuzzy logic systems: introduction and new directions*. Upper Saddle River: Prentice Hall PTR, 2001.

PAGE, M. J. et al. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, v. 372, n. 71, 2021. DOI: 10.1136/bmj.n71.

PM4PY DEVELOPMENT TEAM. PM4Py: Process Mining for Python — Documentation. Fraunhofer Institute for Applied Information Technology, 2024. Disponível em: https://pm4py.fit.fraunhofer.de/. Acesso em: 26 abr. 2026.

QIAN, A. 3 ways AI is transforming BPM. *Perficient Blog*, 2017. Disponível em: https://blogs.perficient.com/. Acesso em: 26 abr. 2026.

RAY DOCUMENTATION. RLlib: scalable reinforcement learning. Anyscale, 2024. Disponível em: https://docs.ray.io/en/latest/rllib/. Acesso em: 26 abr. 2026.

SCHRITTWIESER, J. et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, v. 588, n. 7839, p. 604-609, 2020. DOI: 10.1038/s41586-020-03051-4.

SCHULMAN, J.; WOLSKI, F.; DHARIWAL, P.; RADFORD, A.; KLIMOV, O. Proximal policy optimization algorithms. *arXiv preprint* arXiv:1707.06347, 2017.

SILVA, E.; MARREIROS, G. Predictive process mining: a systematic literature review. In: ROCHA, Á. et al. (eds.). *Good Practices and New Perspectives in Information Systems and Technologies*. WorldCIST 2024. Lecture Notes in Networks and Systems, v. 987. Cham: Springer, 2024. p. 357-378. DOI: 10.1007/978-3-031-60221-4_35.

SILVANDER, J. Business process optimization with reinforcement learning. In: SHISHKOV, B. (Ed.). *Business Modeling and Software Design*. BMSD 2019. Lecture Notes in Business Information Processing, v. 356. Cham: Springer, 2019. p. 203-212. DOI: 10.1007/978-3-030-24854-3_13.

SOLIMAN, G.; MOSTAFA, K.; YOUNIS, O. Reinforcement learning for process mining: business process optimization. In: ROCHA, Á. et al. (eds.). *Good Practices and New Perspectives in Information Systems and Technologies*. WorldCIST 2024. Lecture Notes in Networks and Systems, v. 989. Cham: Springer, 2024. p. 108-125. DOI: 10.1007/978-3-031-60227-6_10.

SOUTO ANIDO, L.; GARCÍA RONDÓN, I. Modelo de gestión de los recursos humanos con base en la teoría de los subconjuntos borrosos. *GECONTEC: Revista Internacional de Gestión del Conocimiento y la Tecnología*, v. 4, n. 2, p. 14-34, 2016.

SUTTON, R. S.; BARTO, A. G. *Reinforcement Learning: an introduction*. 2. ed. Cambridge: MIT Press, 2018.

TENSORFLOW GNN. Graph Neural Networks in TensorFlow. Google Research, 2024. Disponível em: https://blog.tensorflow.org/. Acesso em: 26 abr. 2026.

VAN DER AALST, W. M. P. *Process mining: data science in action*. 2. ed. Berlin: Springer, 2016.

VELIČKOVIĆ, P.; CUCURULL, G.; CASANOVA, A.; ROMERO, A.; LIÒ, P.; BENGIO, Y. Graph attention networks. In: *International Conference on Learning Representations* (ICLR 2018), 2018.

WORKFLOW MANAGEMENT COALITION. WfMC Standards. Lighthouse Point: WfMC, 2018. Disponível em: http://www.wfmc.org/. Acesso em: 26 abr. 2026.

YIN, R. K. *Estudo de caso: planejamento e métodos*. 6. ed. Porto Alegre: Bookman, 2018.

ZHANG, Y.; HOU, Y.; TANG, B.; CHEN, S.; ZHANG, M.; DONG, X.; CHEN, S. GNNs as predictors of agentic workflow performances. *arXiv preprint* arXiv:2503.11301, 2025.

ZHENG, K.; YU, S.; CHEN, B. Ci-GNN: a Granger causality-inspired graph neural network for interpretable brain network-based psychiatric diagnosis. *Neural Networks*, v. 172, art. 106147, 2024. Elsevier. DOI: 10.1016/j.neunet.2024.106147.